

IMS: Towards Computability and Dynamicity for Intent-Driven Micro-Segmentation

Zixuan Ma , Chen Li , Yuqi Zhang , Ruibang You , and Bibo Tu 

Abstract—Micro-segmentation (MSG), a pillar of Zero-Trust, provides fine-grained access control for east-west traffic between cloud endpoints (VMs/containers). Admins formulate strict whitelisting MSG policies that allow necessary traffic. However, current MSG systems lack the computability foundation to resolve policy inconsistencies, where policy overlap can cause conflicts that violate the security requirements, and to verify policy reachability to avoid erroneously blocking necessary traffic. Meanwhile, current MSG systems lack comprehensive dynamicity processing, including maintaining invariants when updating MSG policies and promptly adjusting policy enforcement for endpoint status changes. We propose IMS, the first intent-driven MSG system towards computability and dynamicity. IMS innovatively defines the endpoint group space and algebra, providing the computability foundation for formally and automatically verifying and processing MSG policies. Based on this, IMS implements functionalities to resolve policy inconsistencies and to verify policy reachability. Meanwhile, IMS achieves comprehensive and prompt dynamicity processing. IMS fulfils the verification and dynamicity processing requirements of intent-driven systems. We implement a prototype and evaluations show that the processing time of IMS functionalities scales linearly with the number of policies, and the average endpoint dynamicity processing time is 5.05 ms in the setup of 1,000 endpoints, illustrating that IMS is scalable and can process dynamicity promptly.

Index Terms—Micro-segmentation, zero-trust, policy verification, dynamicity processing, intent-driven system.

I. INTRODUCTION

ZERO-TRUST [1], [2] is a cybersecurity principle that has recently received considerable attention. The core ideas of Zero-Trust are: 1) *Never Trust, Always Verify*: avoid automatic trust and strictly verify all access from anywhere; 2) *Apply Least-Privilege Access*: restrict any entity to the minimum necessary resources; 3) *Assume Breach*: assume that breaches are inevitable, thus always maintain strict access controls and preparedness for threats. Micro-segmentation (MSG) is both a pillar of Zero-Trust and an important technical practice of Zero-Trust that is widely used in cloud computing [3]. The virtual network (VNet) in the cloud is a flat network for east-west traffic that

allows VMs or containers (commonly referred to as endpoint, EP) to freely access each other. Such a VNet is insecure: compromised EP can attack other EPs via the unrestricted network (i.e., lateral movement [4], [5]). Therefore, VNet needs flexible and EP-level access control. MSG is proposed with the following key features to practice Zero-Trust security for VNet: 1) EP-level whitelisting access control, where any access requires explicit permission from MSG policies; 2) formulating strict policies to allow only necessary traffic; and 3) on-demand updating policies in response to threats. These features are consistent with the core ideas of Zero-Trust. Currently, MSG has become one of the most important and fundamental technologies for cloud security [3].

As an emerging security technology, MSG can be divided into three categories based on its evolutionary trajectory: 1) IP-based MSG [6], [7], [8]. The initial MSG method, which requires admins to directly formulate complex network access control rules (e.g., OpenFlow [9] flow rules) for each EP based on its IP and port to implement MSG. However, with the increasing number and dynamicity of EPs in VNet, IP-based MSG has serious problems in policy formulation such as overwhelming number, poor readability and poor maintainability, which makes it difficult to use in practice. 2) Group-based MSG [10], [11]. EPs with the same access control requirements are abstracted into groups and MSG policies are formulated on a group basis, which reduces the number of policies, but still has the problems of readability and maintainability since policies cannot express highly abstract semantics such as business attributes. 3) Label-based MSG [12], [13], [14], [15], [16]. EPs are tagged with labels that express highly abstract semantics (e.g., business attributes of EPs) and MSG policies are formulated through labels and have highly abstract semantics (e.g., *Web* can access *Database* via TCP, 3306), which can resolve the above problems to some extent. As the most advanced method currently, label-based MSG is widely used as a paradigm (e.g., Kubernetes (K8S) Network Policy [12]). However, current label-based MSG still faces the following critical problems.

1) *Policy inconsistency*: Policy overlap can cause conflicts that violate admin's security requirements. For example, for two policies, Policy **A**: Only EPs with the label *Normal* can access each other; Policy **B**: EPs with the label *Web* can access EPs with the label *Database*. For security considerations, admin requires that the access scope allowed by **A** should not be erroneously extended, otherwise EPs without the label *Normal* (e.g., compromised EPs) can access EPs with the label *Normal*, causing **A** to fail and introducing potential attack surface. Other policies should follow the security constraint of **A**. However,

Manuscript received 23 March 2023; revised 8 April 2024; accepted 30 May 2024. Date of publication 13 June 2024; date of current version 16 January 2025. This work was supported by Guangdong Province Key Area Research and Development Program under Grant 2019B010137002. (Corresponding author: Bibo Tu.)

The authors are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: mazixuan@iie.ac.cn; lichen@iie.ac.cn; zhangyuqi@iie.ac.cn; youruibang@iie.ac.cn; tubibo@iie.ac.cn).

Digital Object Identifier 10.1109/TDSC.2024.3413752

current label-based MSG methods [12], [13], [14], [15], [16] cannot detect and process policy inconsistencies: Even if **A** and **B** are correctly formulated, the security requirement is violated as **B** overlaps **A**: EPs with labels *Web* and *Compromised* can access EPs with labels *Normal* and *Database* via Policy **B**.

2) *Reachability violation*: Current label-based MSG methods [12], [13], [14], [15], [16] lack the formalized reachability verification means to automatically verify that MSG policies meet specified reachability requirements and to fix them. Therefore, incorrect/incomplete whitelisting MSG policies without the necessary reachability verification will erroneously block necessary traffic, leading to serious consequences such as business disruption. Taking K8S Network Policy [12] as an example, access from EP_a to EP_b must ensure that the specified traffic can flow out from EP_a and can flow into EP_b . If the policy is incorrect/incomplete at either end, the desired reachability is violated. The lack of automated verification means force admins to manually check all policies one by one for each specified reachability requirement and try to fix them, which is almost impossible in practice for the following reasons: 1) Inefficiency: Manual processing is time-consuming and inefficient for modern clouds with a large number of MSG policies. 2) Complexity: Check and fix can involve multiple policies to be considered simultaneously. 3) Correctness: Manual processing cannot guarantee the correctness.

Due to the above two problems, current label-based MSG methods [12], [13], [14], [15], [16] can neither prevent policies from failing due to overlap to minimize the attack surface, nor ensure the necessary reachability to avoid business disruption. We address these problems by proposing *computability* for label-based MSG. We define computability as a capability that provides a formal representation model of MSG policies, structured in a way that can be automatically processed by the program, and on top of that, provides computational power to enable on-demand policy verification and processing. For policy inconsistencies, the capability to accurately represent and compute possible policy overlaps and to decompose the overlaps to process conflicts is required. For reachability violations, the capability to verify if the policies formally cover the reachability requirements and to implement accurate fix via computation is required. Furthermore, other policy verification and processing functionalities can be implemented on-demand based on such computability, providing additional benefits.

In addition, current label-based MSG methods [12], [13], [14], [15], [16] lack comprehensive dynamicity processing. Dynamic adjustments to business and EPs are common in the cloud, requiring the MSG system to cope with such dynamic changes. Therefore, we propose *dynamicity* for label-based MSG, and since MSG policies and subject/object EPs are the main parts of MSG [17], we divide the definition of dynamicity into two aspects: 1) Policy dynamicity: MSG policies need to be dynamically adjusted to meet new business and security requirements while maintaining specified invariants, e.g., maintaining existing access of specified services to avoid business disruption. This requires computability for formal policy computation, which is lacking in current label-based MSG methods [12], [13], [14], [15], [16]. 2) EP dynamicity: The dynamic creation/destruction

and status changes of EPs require prompt adjustment of the MSG policy enforcement to adjust the allowed access of EPs, otherwise it will affect business operations or pose security threats, e.g., when detected as compromised, EPs may already be attacking other EPs [4], [5]. However, current label-based MSG methods [12], [13], [14], [15], [16] do not consider this problem and do not optimize the processing efficiency of EP dynamicity.

Currently, policy management has become the largest source of cost to organizations in implementing Zero-Trust [18]. Therefore, the evolutionary trajectory of MSG has always been intent-driven, from the initial IP-based MSG to today's label-based MSG, which tries to provide the high abstraction policy that to some extent expresses the admin's intent, helping to better formulate and maintain policies in large and highly dynamic clouds. However, considering the above problems of policy inconsistency, reachability violation, and lack of comprehensive dynamicity processing, current label-based MSG methods [12], [13], [14], [15], [16] are still insufficient. In fact, addressing the above problems coincides with the goal of the intent-driven system [19], whose basic requirements include, in addition to the high abstraction policy and its enforcement, the ability to provide policy verification, conflict processing and dynamicity processing, finally forming a complete closed-loop. Therefore, the radical improvement of the label-based MSG can be considered to make it compatible with the requirements of the intent-driven system to address the above problems.

In this paper, we propose IMS, the first intent-driven MSG system that proposes two novel and important features for label-based MSG: computability and dynamicity. For computability, IMS innovatively defines the endpoint group (EPG) space and algebra, which provides the computable foundation for the formalized implementation of automatically verifying and processing MSG policies. Based on this, IMS implements functionalities to detect and process policy inconsistencies, and to enable reachability verification and automatic fix. For dynamicity, IMS considers it at the policy level and the EP level, respectively. First, based on the computability above, IMS enables the automatic maintenance of the desired invariants as policies are dynamically updated. Second, for EP dynamicity, IMS enables prompt processing through several optimizations to ensure that the admin's security intents remain in force.

Based on these features and functionalities, IMS implements a complete intent-driven MSG system, providing a new paradigm for MSG policy formulation and maintenance. First, admins can formulate label-based MSG policies according to business and security requirements, with the ability to express additional security constraints. Any policy inconsistencies are automatically detected and processed by IMS, ensuring that policies do not fail due to overlap to avoid introducing potential attack surface. Second, admins can ensure that the desired reachability is not violated via reachability verification and automatic fix. Based on the above process, MSG policies can be formulated and verified to have the minimum attack surface without violating the necessary reachability. Finally, IMS is able to maintain specified invariants as admins update policies to avoid impacts on business, and is able to automatically keep MSG policies in

force continuously as EP dynamicity occur. IMS provides an intent-driven closed-loop through label-based policy formulation, policy verification, conflict processing and dynamicity processing. Before IMS is proposed, formulating and maintaining proper MSG policies that minimize the attack surface without disrupting business traffic was a difficult or even impossible task for admins.

Our contributions are as follows.

- 1) We propose IMS, the first intent-driven MSG system towards computability and dynamicity. IMS innovatively defines the EPG space and its algebra, laying the foundation for computability (Section IV). Based on this computability, IMS implements functionalities to resolve policy inconsistencies and reachability violations. IMS takes full account of dynamicity, enables automatic maintenance of desired invariants when updating policies, and enables prompt EP dynamicity processing through optimizations. Based on the above features, IMS realizes a complete intent-driven closed-loop for MSG.
- 2) We design the label system of IMS to enable the label-based formulation of MSG policies. Based on this, we design the Domain-Specific Language (DSL) of IMS to express MSG policies and functionalities, including policy inconsistency detection and processing, reachability verification and fix, and dynamicity processing. We implement these functionalities, and perform optimizations to ensure the scalability of these functionalities and the prompt processing of dynamicity (Section V).
- 3) We implement a prototype of IMS based on SDN, enabling MSG policy enforcement through the SDN controller and software switch. IMS is highly adaptable and can enable MSG in both VM clouds and container clouds. We build a use case based on this prototype to show how IMS is used (Section VI). Performance evaluations conducted on the prototype show that the functionalities of IMS are scalable in performance, and that IMS can process EP dynamicity promptly (Section VII).

We describe the background, motivation and overview in Sections II and III. We discuss limitations and future work in Section VIII. We describe related work and conclude in Sections IX and X.

II. BACKGROUND AND MOTIVATION

A. Micro-Segmentation

In the private cloud or the virtual private cloud (e.g., Amazon VPC[20]), EPs within the same VNet can freely access each other without any necessary access control, leading to security problems. For example, assuming a ransomware virus has compromised an EP, the typical attack flow [21] is as follows: Through the unrestricted VNet, the compromised EP first obtains the IPs of other EPs in the VNet through port scanning, and then exploits other EPs' vulnerable ports to spread the infection. This cycle repeats, allowing the ransomware to compromise as many EPs in the VNet as possible.

Therefore, MSG is proposed with the following key *Features* to practice Zero-Trust security for VNet: 1) EP-level whitelisting

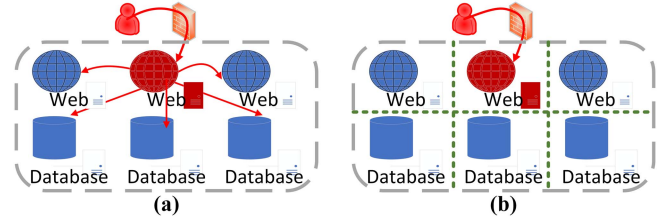


Fig. 1. An example of label-based micro-segmentation.

access control, where any access requires explicit permission from MSG policies; 2) formulating strict policies to allow only necessary traffic; and 3) on-demand updating policies in response to threats. Here is an example of how MSG realizes the above features. When MSG is not enabled (Fig. 1(a)), access within the VNet is unrestricted. Once the *Web* EP is compromised, all EPs can be attacked through lateral movement. When MSG is enabled (Fig. 1(b)), fine-grained access control (green lines in Fig. 1(b)) can be provided at EP-level. Any access must be explicitly allowed by the MSG policy (*Feature 1*). Admins follow *Feature 2* by formulating label-based policies that allow only necessary business traffic in the form of “*Web* can access *Database* via TCP, 3306”. MSG policies will be compiled into rules acceptable to VNet (e.g., OpenFlow [9] flow rules) and enforced. Since only specified traffic is allowed between specified EPs, the attack surface is effectively reduced. Finally for *Feature 3*, admins can add/update MSG policies to always keep the attack surface smallest possible as business adjustments occur.

B. Motivation

Constraint-Based Policy Composition: MSG policies are whitelisting policies that contain only *allow* actions, so there is no explicit conflict between *deny* and *allow* actions as in blacklisting policies. However, policy overlap can still lead to implicit policy inconsistencies (conflicts) that violate security requirements. We illustrate this with a realistic example. Suppose that there are EPs that provide *Web* (*W*) and *Database* (*D*) services (with *W* and *D* labels, respectively). As business requirements, *W* can access *D* (Policy A). Especially, some of these EPs belong to *Payment* (*P*) service (with *P* label). As the security isolation requirement, *D* in *P* service (*PD*) can *only* be accessed by *W* in *P* service (*PW*) (Policy B), which actually imposes the additional constraint and requires strict adherence by other policies. However, using current label-based MSG methods [12], [13], [14], [15], [16], even if policies A and B are correctly formulated, the security requirement is still violated due to the policy overlap between policies A and B: EPs with label *P* intersect with EPs with label *W* and EPs with label *D*. Therefore, Policy B is overlapped by Policy A. The result is shown in Fig. 2(a): *PD* can be accessed by *W*, which violates the security isolation requirement of Policy B. Policy overlap can also lead to redundancy, which seriously affects the efficiency of policy compilation and enforcement.

It is challenging to compose policies in which all conflicts due to overlaps are processed. One EP can be acted upon by multiple

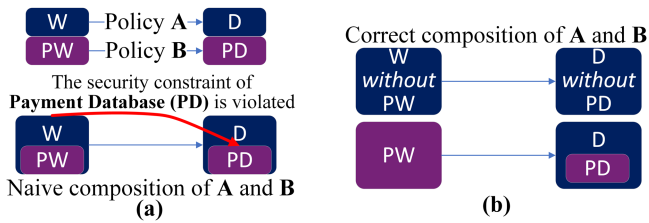


Fig. 2. An example of MSG policy composition.

policies, resulting in complex overlaps between them. Such overlap can be explicit, as in the example in Fig. 2, where policies have duplicate labels. It can also be implicit, as in the example of policy inconsistency given in Section I, where policies do not have duplicate labels. Fig. 2(b) shows the correct composition that maintains the constraint of Policy B. Policy composition requires the formal representation and automatic computation of policy overlap and the policy adjustment based on the constraint, which cannot be achieved by current label-based MSG methods [12], [13], [14], [15], [16]. Without automated means, admins need to manually check and adjust policies to ensure consistency, which is inefficient, difficult, and error-prone and becomes infeasible as the number of EPs increases and inter-access becomes more complex. Therefore, the MSG system should be able to formally represent and automatically compute policy overlaps, and on this basis support constraint-based policy composition to automatically processes any conflicts.

Policy-Level Reachability Verification: Current label-based MSG methods [12], [13], [14], [15], [16] lack formalized and automated means to verify and fix the reachability before policies take effect. This raises serious problems: Since any access must be explicitly allowed by MSG policies and manually formulated policies are not immune to errors/omissions, incorrect/incomplete MSG policies without the necessary reachability verification will erroneously block necessary traffic, leading to serious consequences such as business disruption. The lack of automated verification means forces admin to manually check policies, which is inefficient, difficult and error-prone. Another solution provided by current MSG methods is to directly and manually observe inter-EP accesses in a simulated/real environment to verify the reachability [14]. This is not only inefficient and costly, but more importantly, this solution verifies reachability at the EP level, which has a semantic gap with MSG policies: Even if reachability is satisfied in the current VNet, it does not mean that reachability is satisfied at the policy level, e.g., when new EPs are launched with labels that have not been previously observed, their access can still be erroneously blocked, and repeating the above verification is too costly.

Therefore, there is an urgent need to verify reachability at policy-level. When the admin specifies the desired reachability to be verified, the MSG system should be able to verify whether the current policies formally cover the reachability requirements and to implement automatic fix via computation.

Computability: To implement constraint-based policy composition and policy-level reachability verification, we are inspired by HSA [22], a physical network verification method

that defines the formal representation and algebra of the packet header field, and implements verification procedures based on this algebra. HSA provides us with the key insight: trying to formally represent MSG policies and define algebraic operations to automate policy verification and processing. Therefore, we propose a new feature for label-based MSG, computability, which is defined as a capability to represent MSG policies in computable formal structures and define algebraic operations in order to achieve on-demand policy processing and verification and to automate them programmatically. Specifically, we implement computability in IMS by defining the EPG space to formally represent label-based MSG policies, and defining the EPG space algebra for on-demand policy verification and conflict processing. Based on this computability, IMS implements constraint-based policy composition and policy-level reachability verification. For example, based on the EPG space and algebra, IMS can formally compute the *intersection* of policies A and B in Fig. 2 to represent their overlap, and compute the *difference* sets between A and B and this *intersection* respectively to adjust them, and thus programmatically automating policy composition.

Dynamicity: Cloud computing enables rapid business iteration and dynamic adjustment of EP, requiring MSG systems to cope with such dynamic changes. Therefore, we propose a new feature, dynamicity, for label-based MSG. As an access control method, MSG follows the access control triad model (S, O, A) [17] and consists of MSG policies and subject/object EPs, where MSG policies give *Subject* (S) EPs access *Right* to *Object* (O) EPs and form the matrix A . Therefore, we divide the definition of dynamicity into two parts:

- 1) **Policy dynamicity:** MSG policies need to be dynamically adjusted as business and security requirements change. Adjustments need to meet new requirements without impacting existing services to avoid business disruption. Therefore, policy dynamicity is defined as a capability to enable admins to dynamically adjust MSG policies and to automatically maintain the required invariants (e.g., reachability of specified service) via formal policy computation based on computability during adjustment. However, current label-based MSG methods [12], [13], [14], [15], [16] lack the computability to achieve policy dynamicity.
- 2) **EP dynamicity:** The dynamic creation/destruction and status changes of EPs will affect MSG policy enforcement, e.g., consider Policy A in Section I, when the status of an EP changes from normal to compromised, the allowed access of it should be adjusted. Adjustments should be as prompt as possible to 1) minimize the impact on business operations, e.g., to minimize the latency due to MSG in establishing traffic for EPs with changed status, and 2) minimize security threats, e.g., when detected as compromised, EPs will rapidly try to or are already attacking other EPs via lateral movement [4], [5]. Therefore, EP dynamicity is defined as a capability to promptly adjust the MSG policy enforcement for dynamically changing EPs. However, current label-based MSG methods [12], [13], [14], [15], [16] do not consider EP dynamicity and optimize accordingly.

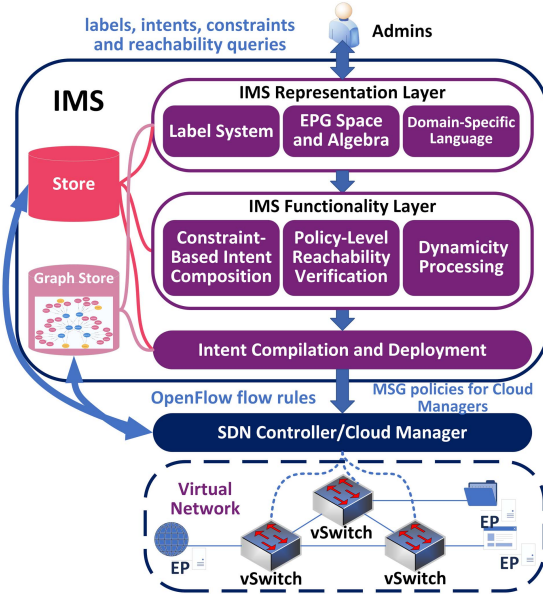


Fig. 3. IMS architecture.

Based on the EPG space and algebra, IMS implements the functionality to automatically maintain the desired invariants when MSG policies are dynamically adjusted. Meanwhile, IMS processes policy redundancy and proposes several optimizations to achieve prompt EP dynamicity processing.

III. IMS OVERVIEW

The system architecture of IMS is shown in Fig. 3. The main components and workflow are as follows.

IMS Representation Layer: This layer consists of three main parts. 1) *Label system* (Section IV-A), where labels can be specified according to actual needs. Labels are used to tag EPs and formulate policies. 2) *EPG space and algebra* (Sections IV-B and IV-C). This part is the basis for the computability, based on which IMS implements its functionality layer. 3) *Domain-specific language (DSL)* (Section V-A). IMS designed the DSL to formulate, compose, verify and dynamically modify policies.

IMS Functionality Layer: This layer consists of three main functionalities of IMS. 1) *Constraint-based intent composition* (Section V-B). IMS treats ordinary MSG policies as intents and designs additional constraints. In this paper, intents and constraints are collectively referred to as policies. Given sets of intents and constraints, IMS automatically detects and processes any conflicts in them and obtains conflict-free sets of intents and constraints as the composition result. 2) *Policy-level reachability verification* (Section V-C). Given the desired reachability to be verified, IMS verifies that the composition result satisfies the desired reachability, and automatically fixes the reachability according to the admin's command. 3) *Dynamicity processing* (Section V-E). For policy dynamicity, where policies are dynamically modified via DSL, IMS automatically maintains the required invariants. For EP dynamicity, IMS achieves prompt processing through several optimizations.

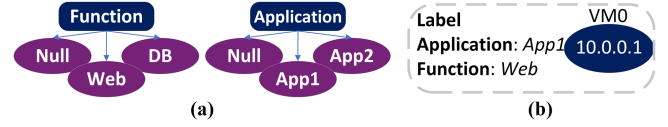


Fig. 4. (a) An example of the label forest; (b) An examples of EP's labels.

Intent compilation and deployment (Section V-D). IMS uses the data plane information to compile the MSG intents into policies acceptable to the data plane, and to select appropriate policy enforcement points for EPs to deploy the policies.

Store: In addition to storing the input, result and data plane information, IMS also uses a graph database to store and display the access between EPs allowed by intents.

IMS architecture: IMS provides the interface upward, deploys the compiled policies for the data plane through the SDN controller or the cloud manager downward, and meanwhile obtains data plane information. In this paper, for the adaptability to different types of clouds, IMS uses the SDN controller by default to realize MSG through OpenFlow [9] flow rules. However, IMS can also interact directly with cloud managers such as OpenStack [23] and Kubernetes [24].

IV. IMS LABEL SYSTEM AND ALGEBRA

A. Label System

IMS uses labels to realize the intent-based representation of MSG policies, including the following parts.

Label Tree (LT): A two-level multi-way tree. Label type and label values are the root node and leaf nodes respectively. Leaf nodes at the same level naturally have mutually exclusive relationships, so restrictions can be imposed on the formulation of label values to avoid semantic duplication and coverage.

Label Forest (LF): IMS specifies that the label types of LTs cannot be repeated, so several different LTs will form an LF. Fig. 4(a) shows an example of an LF consisting of two LTs. It can be seen that the formulation of the LF is closely related to high-level business and security requirements. As a unified perspective for formulating labels, the LF is used by admins in the form of interfaces. On the premise of abiding by the relevant regulations of IMS on the LF, any label types and values can be formulated according to actual needs, which greatly improves adaptability and expressiveness.

Labels are tagged for each EP through IMS to express the high-level attributes of the EP. As shown in Fig. 4(b), IMS retrieves and stores the low-level attributes (e.g., IP address) of VM_0 from the data plane, which will uniquely identify the EP within the VNet. VM_0 is also tagged with labels according to the LF in Fig. 4(a) to express its high-level attributes. Therefore, the high-level attributes of EPs will replace the low-level attributes to complete the MSG policy formulation.

B. EPG Space

After introducing the label system, we will introduce the concept of endpoint group (EPG). As EPs are tagged with labels,

a group of EPs can be selected by specifying labels, called EPG. EPG is a fundamental component of any policy in IMS and is used to delineate the scope of the EPs involved in the policy. Specifically, an EPG consists of ≥ 1 label(s), and each label consists of ≥ 1 label value(s). The relationship between different labels is *and*, while the relationship between the values of the same label is *or*. For example, if an EPG is “*Function: Web; Application: APP1, APP2*”, it means that all the EPs as *Web* in *APP1* or in *APP2* are selected.

We further formally define the EPG space $\mathcal{E}$. Specifically, $\mathcal{E}$ is represented by k types of labels, where each type of label contains several label values, so it can be expressed as a disjunction form $label_i = \bigvee_{0 \leq j < n_i} (b_{i,j})$, where n_i represents the number of values of the i -th label, $b_{i,j} \in \{0, 1\}$, and 1/0 represents whether to select the j -th value of the i -th label. Therefore, $\mathcal{E}$ can be represented as a conjunction form consisting of disjunctions $\mathcal{E} = \bigwedge_{0 \leq i < k} (\bigvee_{0 \leq j < n_i} (b_{i,j}))$.

C. EPG Space Algebra

In this section, we introduce the EPG space algebra, which enables the computability of IMS. First, we need to resolve the practical problem of how to map a given EPG to the full EPG space $\mathcal{E}$. Specifically, an EPG does not necessarily contain all of the label types in the label forest. For example, a policy may be formulated using only some, rather than all, of the label types. However, such an EPG cannot be mapped directly into $\mathcal{E}$ because some dimensions (label types) are missing. Therefore, IMS needs to perform the following preprocessing.

- 1) For each label type, it is specified that each EP must be tagged with a label value. In practice, EPs may not need or have certain label types, or the label values are currently ambiguous/unknown. Therefore, IMS sets the first value of each LT to *Null* by default. For the above cases, the label value of corresponding label type of the EP is tagged as *Null*.
- 2) For each EPG, IMS inserts any label type that is not contained in it into its conjunction form, and $b_{i,j}$ in it is all set to 1, i.e., all values of this label type are selected by default. The intuitive explanation is that since EP has a value for each label type, if some label types are not specified in EPG, all values under those label types are actually selected by default.

After the above preprocessing, any EPG can be mapped to the EPG space $\mathcal{E}$. Then we can define set operations on $\mathcal{E}$ to implement requirements such as the computation of overlap between policies. To be intuitive and easy to compute programmatically, the EPG space $\mathcal{E}$ needs to be transformed into a formally computable set. Specifically, for each $label_i$ in $\mathcal{E}$, choose one $b_{i,j}$ with value of 1 in order, and use it to combine into a conjunction $e^{min} = \bigwedge_{0 \leq i < k} b_i$, where $b_i \in label_i$ and $b_i = 1$. e^{min} is the smallest granularity element. For any two e^{min} , they are either completely equal or mutually exclusive. Thus, $\mathcal{E}$ is transformed into the set form $\mathcal{E} \equiv \bigcup e^{min}$ containing mutually exclusive elements e^{min} .

After the transformation, we can conveniently define set operations on $\mathcal{E}$. For two EPGs $E_i, E_j \in \mathcal{E}$, define the set operation

TABLE I
SYMBOL, ACRONYM AND DEFINITION

Symbol & Acronym	Definition
$E \iff EPG$	Specify a group of EPs via labels
$eE \iff egressEPG$	EPG for the egress direction
$iE \iff ingressEPG$	EPG for the ingress direction
e^{min}	Smallest element of EPG
$e^{e,min}$	e^{min} for the egress direction
$e^{i,min}$	e^{min} for the ingress direction
$R(s) \iff Relation(s)$	Specify the type(s) of traffic
$I \iff Intent$	Consist of eE , Rs and iE
$C \iff Constraint$	Consist of E , eRs and iCs
$eC(s) \iff egressConstraint(s)$	Constraint(s) on the egress of E
$iC(s) \iff ingressConstraint(s)$	Constraint(s) on the ingress of E
$dd \iff defaultdeny$	Further semantic for <i>constraint</i>
$RQ \iff ReachabilityQuery$	Consist of eE , Rs and iE
Φ_I, Φ_C, Φ_R	Sets of I , C and RQ
$\Phi_{M,(E)}, \Phi_{D,(E)}$	Sets of $I(E)$ to maintain/delete
$\Phi_{A,C}$	Set of C to add
Φ_E	Set of endpoints

between them as follows.

$$\begin{aligned}
 E_i \cap E_j &= \{\forall e^{min} | (e^{min} \in E_i) \wedge (e^{min} \in E_j)\} \\
 E_i \cup E_j &= \{\forall e^{min} | (e^{min} \in E_i) \vee (e^{min} \in E_j)\} \\
 E_i - E_j &= \{\forall e^{min} | (e^{min} \in E_i) \wedge (e^{min} \notin E_j)\} \\
 E_i \subseteq E_j &= \begin{cases} True, & \text{if } E_i \cap E_j = E_i \\ False, & \text{if } E_i \cap E_j \neq E_i \end{cases}
 \end{aligned}$$

Suppose that the number of label types is 3 and the number of values of each label is 2, $E_a = (b_{0,0}) \wedge (b_{1,0})$, $E_b = (b_{1,0}) \wedge (b_{2,0})$, $b_{i,j}$ are all 1. After preprocessing, $E_a = (b_{0,0}) \wedge (b_{1,0}) \wedge (b_{2,0} \vee b_{2,1})$, $E_b = (b_{0,0} \vee b_{0,1}) \wedge (b_{1,0}) \wedge (b_{2,0})$. After the transformation to the set form, $E_a = \{(b_{0,0} \wedge b_{1,0} \wedge b_{2,0}), (b_{0,0} \wedge b_{1,0} \wedge b_{2,1})\}$, $E_b = \{(b_{0,0} \wedge b_{1,0} \wedge b_{2,0}), (b_{0,1} \wedge b_{1,0} \wedge b_{2,0})\}$. At this time it is convenient to perform set operations on them. For example, their intersection is $E_a \cap E_b = \{(b_{0,0} \wedge b_{1,0} \wedge b_{2,0})\}$.

V. DETAILED DESIGN

This section describes the detailed design of IMS. We first introduce the Domain-Specific Language (DSL) in the representation layer, which defines the syntax and usage of primitives in IMS (Section V-A). Based on the representation layer, IMS implements its functionality layer, including constraint-based intent composition (Section V-B), policy-level reachability verification (Section V-C), and dynamicity processing (Section V-E). We also describe intent compilation and deployment in IMS (Section V-D). Finally, we describe the optimizations for the scalability of functionalities and prompt dynamicity processing (Section V-F). Table I lists the symbols and their acronyms and definitions.

A. Domain-Specific Language (DSL) in IMS

In this DSL (defined via EBNF [25]), IMS defines the syntax and structure of *intent*, *constraint*, and *reachability query*,


```

<functionality> ::= <compose> | <verify> | <modify>
<compose> ::= 'compose:' <constraints> <constraints>; 'intents:' <intents>
<verify> ::= 'verify:' <RQs> ['fix']
<modify> ::= 'add:' <add> | 'delete:' <delete>; 'maintain:' <maintain>
<add> ::= 'constraint:' <constraints> | 'intent:' <intents>
<delete> ::= 'EPG:' <EPGs> | 'intent:' <intents> | 'constraint:' <constraints>
<maintain> ::= 'EPG:' <EPGs> | 'intent:' <intents> | 'original'
<intent> ::= 'egress EPG:' <EPG> \n ingress EPG: <EPG>
          \n relations: <n' <relations>
<intents> ::= <intent> {'\n' <intent>}
<RQ> ::= 'egress EPG:' <n' <EPG> \n ingress EPG: <n' <EPG>
        \n relations: <n' <relations>
<RQs> ::= <RQ> {'\n' <RQ>}
<constraint> ::= 'EPG:' <n' <EPG>
               [\n egress constraint: <n' <eCs>]
               [\n ingress constraint: <n' <iCs>]
<constraints> ::= <constraint> {'\n' <constraint>}
<eCs> ::= (<eC> {'(' | ',' | '\n' <eC>}) (' | ',' | '\n') ['dd:' <dd>]
<iCs> ::= (<iC> {'(' | ',' | '\n' <iC>}) (' | ',' | '\n') ['dd:' <dd>]
<eC> ::= 'relation:' <relation> \n ingress EPG: <n' <EPG>
<iC> ::= 'egress EPG:' <n' <EPG> \n relation: <relation>
<EPG> ::= <label> {'\n' <label>}
<EPGs> ::= <EPG> {'\n' <EPG>}
<label> ::= <labelType> ':' <labelValues>
<labelType> ::= labelType
<labelValues> ::= <labelValue> {'(' | ',' | '\n' <labelValue>}
<labelValue> ::= labelValue
<relations> ::= <relation> {'\n' <relation>}
<relation> ::= <protocol> ':' <port>
<protocol> ::= protocol
<port> ::= port
<dd> ::= 'true' | 'false'

```

(a)

(b) **egress EPG:**
Function: Web
ingress EPG:
Function: DB
relations:
TCP: 3306

(c) **EPG:**
Application: App1
Function: DB
ingress constraint:
egress EPG:
Application: App1
Function: Web
relation: TCP: 3306
dd: true

Fig. 5. (a) Domain-Specific Language (DSL) in IMS; (b) An example of *intent*; (c) An example of *constraint*.

and defines primitives *compose*, *verify*, and *modify*, corresponding to the functionalities of the functionality layer respectively. The specific implementations are detailed below.

Intent: *intent* (*I*) (1) consists of *E* (*EPG*) and *Rs* (relations). *E* is represented by labels and classified into *eE* (egress *EPG*) and *iE* (ingress *EPG*) according to the egress and ingress directions of the traffic. *Rs* specifies the types of a set of traffic between *eE* and *iE*. *eE*, *Rs*, and *iE* form a directed RDF graph relationship which means “Allow traffic of the types specified by *Rs* to flow out of *eE* and into *iE*”.

$$I = (eE, Rs, iE) \quad (1)$$

The specific syntax definition of *I* is shown in Fig. 5(a). *EPG* is specified by several *labels*, each *label* contains *labelType* and several *labelValues*. *Rs* consists of several *R* (relation), each *R* specifies the traffic type and contains the matching field, which consists of *protocol* and *port*, indicating the traffic protocol type and port number respectively.

Fig. 5(b) shows an example of the MSG *intent* formulated according to the *LF* in Fig. 4(a), which means that in *App1* and *App2*, *Web* is allowed to access *DB* via TCP 3306.

Constraint: *constraint* (*C*) (2) consists of *E*, *eCs* (egress constraints) (3), and *iCs* (ingress constraints) (4). *E* is the constraint subject of *C*, and *eCs* and *iCs* are constraints on the egress and ingress traffic of *E*.

$$C = (E, eCs, iCs) \quad (2)$$

$$eCs = \left(\bigcup eC, dd \right) = \left(\bigcup (R, iE), dd \right) \quad (3)$$

$$iCs = \left(\bigcup iC, dd \right) = \left(\bigcup (R, eE), dd \right) \quad (4)$$

The specific syntax definition of *C* is shown in Fig. 5(a). *dd* (default deny) is designed to provide further constraint semantics: *false* means that *E* only allows traffic of type *R* originating from *eE* or destined for *iE*, and imposes no restrictions on the traffic of other types; while *true* means that all traffic that is not of *R* type is denied by default. *eCs* consists of a set of *eC* and one *dd*, where *eC* consists of *iE* and *R*. This means that the traffic specified by *R* with *E* as the egress *EPG* can only flow to the specified *iE*. Similarly, *iCs* consists of a set of *iC* and one *dd*, and *iC* means that the traffic specified by *R* with *E* as the ingress *EPG* can only flow from the specified *eE*.

Fig. 5(c) shows an example of *constraint* formulated according to the *LF* in Fig. 4(a), which means that in *App1*, *DB* can and only can receive TCP 3306 traffic sent by *Web*. *constraint* itself does not actively establish any inter-EP connections, but acts on *intent* to prevent it from incorrectly expanding inter-EP access due to overlap, leading to issues such as the violation of admin’s security requirements and privilege leakage. *intent* and *constraint* are used in combination with the primitive *compose* to achieve constraint-based intent composition. For example, using *compose* for the *intent* and *constraint* in Fig. 5 will result in the following: since the *constraint* restricts the exclusivity of the *DB* in *App1*, it can only accept accesses from the *Web* in *App1*. On this basis, access from *Web* to other *DB* is unrestricted.

Reachability Query. *reachability query* (*RQ*) is used to express reachability requirements and is formally identical to *intent*. It is used in combination with the primitive *verify* to verify that current MSG policies meets the desired reachability, and return the unreachable part of it with the causes, which can help admins to judge whether current policies are effective and whether the unreachable part is reasonable, and then determine whether to perform automated reachability fix through *fix*.

One of the typical and practical scenarios using reachability verification is that after security personnel (concerned with least privilege access) have formulated MSG policies, business developers (concerned with business reachability) can use the reachability verification provided by IMS to verify at policy-level that business reachability has not been violated due to errors or omissions and to perform automatic fix to avoid problems such as business disruption. For example, for the composed result in Fig. 5, the reachability of *App2* can be verified in the form “*verify*: [(Function: *Web*, Application: *App2*), TCP 3306, (Function: *DB*, Application: *App2*)]”. IMS will return the deterministic result: the reachability is satisfied.

Modify. **modify** is used for dynamicity processing at the policy level and consists of three primitives *add*, *delete* and *maintain*, where *add* is used to add new *intent* and *constraint*, and *delete* is used to delete *intent* (which can be specified via *EPG*) and *constraint*.

The key primitive is *maintain*, which is used to maintain desired reachability when dynamically updating policies, either by specifying *intent* and *EPG* to maintain partial reachability, or by specifying *original* to maintain full original reachability. For example, maintain full original reachability when adding *constraint* with *add* (which results in performing a new constraint-based intent composition), or specify that the original reachability of some EPs remains unchanged via *EPG* when deleting *intent* with *delete*, to avoid incorrectly affecting existing business when updating policies. For the composed result in Fig. 5, we can use the form “*delete intent*: (Function: *Web*, *, Function: *DB*), *maintain EPG*: (Application: *App1*)” to remove all the access from *Web* to the *DB* but keep all the access of *App1* unchanged.

B. Constraint-based Intent Composition

This section details the implementation of *compose*. Given the set of intents Φ_I and the set of constraints Φ_C , *compose* enables constraint-based intent composition. The processing is sequentially divided into three parts: redundancy processing within Φ_I , conflict processing within Φ_C , and conflict processing between Φ_I and Φ_C .

Redundancy Processing Within Φ_I : Although there are no explicit conflicts between intents, overlaps can also lead to redundancy. Therefore, we first need to process Φ_I to make it has no redundancy. Specifically, eE and iE of each $I \in \Phi_I$ are preprocessed and transformed (Section IV-C) to become an operable set, i.e., $eE = \bigcup e^{e,min}$ and $iE = \bigcup e^{i,min}$. To facilitate subsequent processing, each $e^{e,min}$ in eE is used as new eE to obtain a new $I' = (e^{e,min}, Rs, iE)$, and $I = \bigcup I'$. On top of that, we can further define the redundancy detection and processing between two I' .

Definition 1: For $I'_p = (e_p^{e,min}, Rs_p, iE_p)$ and $I'_q = (e_q^{e,min}, Rs_q, iE_q)$, if $e_p^{e,min} = e_q^{e,min}$ and $Rs_p \cap Rs_q \neq \emptyset$, then there is redundancy between them. The way to process redundancy is to consolidate overlapping elements. According to the intersection between Rs_p and Rs_q , we can get three parts: for $\forall R \in Rs_p - Rs_q$, we have $I'_{p,0} = (e_p^{e,min}, \cup R, iE_p)$; for $\forall R \in Rs_q - Rs_p$, we have $I'_{p,1} = (e_p^{e,min}, \cup R, iE_q)$; for $\forall R \in Rs_p \cap Rs_q$, we have $I'_{p,2} = (e_p^{e,min}, \cup R, iE_p \cup iE_q)$. Finally, we get the merged $I'_p = I'_{p,0} \cup I'_{p,1} \cup I'_{p,2}$.

We process all the $I' \in \Phi_I$ according to Definition 1 to get $\Phi'_I = \bigcup I' = \bigcup (e^{e,min}, \bigcup (R, iE))$ which has no redundancy.

Conflict Processing Within Φ_C : There may be conflicts between constraints in Φ_C . Before processing the conflict between Φ_I and Φ_C , we need to process the conflict within Φ_C . Similarly, E of each $C \in \Phi_C$ is preprocessed and transformed (Section IV-C) to become an operable set, i.e., $E = \bigcup e^{min}$. To facilitate subsequent processing, each e^{min} in E is used as new E to obtain a new $C' = (e^{min}, eCs, iCs)$, and $C = \bigcup C'$. On top of

Algorithm 1: Constraint-Based Intent Composition:

Input: intent set Φ_I and constraint set Φ_C

Output: processed intent set Φ'_I and constraint set Φ'_C

/* Redundancy processing within Φ_I */

1: **for** $I' \in \Phi_I$ **do**

2: Put I' with the same $e^{e,min}$ in the same set $sub\Phi_I$ and get $\Phi_I = \bigcup sub\Phi_I$

3: **for** $sub\Phi_I \in \Phi_I$ **do**

4: Use Definition 1 to process all the I' in $sub\Phi_I$ and merge them one by one to get the final one I' , and finally get processed $\Phi'_I = \bigcup I'$

/* Conflict processing within Φ_C */

5: **for** $C' \in \Phi_C$ **do**

6: Put C' with the same e^{min} in the same set $sub\Phi_C$ and get $\Phi_C = \bigcup sub\Phi_C$

7: **for** $sub\Phi_C \in \Phi_C$ **do**

8: Use Definition 2 to process all the C' in $sub\Phi_C$ and merge them one by one to get the final one C' , and finally get processed $\Phi'_C = \bigcup C'$

/* Conflict processing between Φ'_I and Φ'_C */

9: **for** $I' \in \Phi'_I, C' \in \Phi'_C$ **do**

10: **if** $C'.e^{min} = I'.e^{e,min}$ **then**

11: Use Definition 3 to process I'

12: **return** Φ'_I, Φ'_C

that, we can further define the conflict detection and processing between two C' .

Definition 2: For $C'_p = (e_p^{min}, eCs_p, iCs_p)$ and $C'_q = (e_q^{min}, eCs_q, iCs_q)$, if $e_p^{min} = e_q^{min}$, then there may be conflicts between them. The way to process conflict is to find the intersection of them, i.e., to merge C'_p and C'_q to get the new C'_p . For egress direction, for $\forall i, j$, $eC_{p,i} \in eCs_p$, $eC_{q,j} \in eCs_q$, if $eC_{p,i}.R = eC_{q,j}.R$, then let $eC_{p,i}.iE = eC_{p,i}.iE \cap eC_{q,j}.iE$. After that, processing is different according to the values of dd of eCs_p and eCs_q . If both dd_p and dd_q are *false*, the rest of eCs_q is merged into eCs_p . If dd_p and dd_q are both *true*, then let $\bigcup eC_{p,i}$ replace eCs_p . If $dd_p = \text{true}$ and $dd_q = \text{false}$, no further processing is needed. Then let eCs_p be the result of the merging of eCs_p and eCs_q . The ingress direction is processed in the same way as above. Finally, we get the new C'_p as the result.

We process all the $C' \in \Phi_C$ according to Definition 2 to get $\Phi'_C = \bigcup C' = \bigcup (e^{min}, eCs, iCs)$ which has no conflict.

Conflict Processing Between Φ'_I and Φ'_C : Having obtained Φ'_I which has no redundancy and Φ'_C which has no conflict, we can process the conflict between them.

Definition 3: Intents that violate the constraint should be processed to satisfy the constraint. For $C'_p = (e_p^{min}, eCs_p, iCs_p)$, $I'_q = (e_q^{min}, \bigcup (R_q, iE_q))$. 1) For egress direction, if $e_p^{min} = e_q^{min}$, then there may be conflicts between them. For $\forall i, j$, $eC_{p,i} \in eCs_p$, $(R_{q,j}, iE_{q,j}) \in I'_q$, if $eC_{p,i}.R = R_{q,j}$, then let $iE_{q,j} = iE_{q,j} \cap eC_{p,i}.iE$. In addition, if $dd_p = \text{true}$, and for $\forall j$, $(R_{q,j}, iE_{q,j}) \in I'_q$, if for $\forall i$, $eC_{p,i} \in eCs_p$, $eC_{p,i}.R \neq R_{q,j}$, then remove $(R_{q,j}, iE_{q,j})$ from I'_q . 2) For ingress direction, if $\exists j$, $(R_{q,j}, iE_{q,j}) \in I'_q$, $e_p^{min} \in iE_{q,j}$,

then there may be conflicts between them. For $\forall i, iC_{p,i} \in iCs_p$, if $iC_{p,i}.R = R_{q,j}$ and $e_q^{e,min} \notin iC_{p,i}.eE$, then remove e_p^{min} from $iE_{q,j}$. In addition, if $dd_p = true$, and for $\forall i, iC_{p,i} \in iCs_p$, if $iC_{p,i}.R \neq R_{q,j}$, then remove e_p^{min} from $iE_{q,j}$. Finally, we get the processed I'_q .

We process Φ'_I according to *Definition 3* to make it satisfy Φ'_C and finally get the processed Φ'_I . The overall implementation of constraint-based intent composition is shown in *Algorithm 1*.

C. Policy-Level Reachability Verification

This section details the implementation of *verify*. Given the reachability query (RQ) set Φ_R and the intent set Φ'_I and constraint set Φ'_C after the composition of *compose*, *verify* enables policy-level reachability verification. To facilitate the subsequent processing, we first need to process the redundancy within Φ_R in the same way as we process the intent set Φ'_I and get $\Phi'_R = \bigcup(e^{e,min}, \bigcup(R, iE))$. After that, the processing is divided into two parts: verify if Φ'_C satisfies Φ'_R and fix it; verify if Φ'_I satisfies Φ'_R and fix it. Note that the following procedure performs automatic fix by default when the reachability violation is detected.

Verify if Φ'_C Satisfies Φ'_R and Fix it: For the processed Φ'_R which has no redundancy and the processed Φ'_C which has no conflict, we can verify if Φ'_C satisfies the required reachability of Φ'_R and automatically fix it.

Definition 4: For $C'_p = (e_p^{min}, eCs_p, iCs_p)$, $RQ'_q = (e_q^{e,min}, \bigcup(R_{q,j}, iE_{q,j}))$. 1) For egress direction, if $e_p^{min} = e_q^{e,min}$, then there may be reachability violation. For $\forall i, j, eC_{p,i} \in eCs_p, (R_{q,j}, iE_{q,j}) \in RQ'_q$, if $eC_{p,i}.R = R_{q,j}$ and $iE_{q,j} \not\subseteq eC_{p,i}.iE$, then compute $iE_{q,j} - eC_{p,i}.iE$ as the part where the reachability is violated and let $eC_{p,i}.iE = eC_{p,i}.iE \cup iE_{q,j}$ for fix. In addition, if $dd_p = true$, and for $\forall j, (R_{q,j}, iE_{q,j}) \in RQ'_q$, if for $\forall i, eC_{p,i} \in eCs_p, eC_{p,i}.R \neq R_{q,j}$, then $(e_q^{e,min}, R_{q,j}, iE_{q,j})$ is the part where the reachability is violated and added to eCs_p for fix. 2) For ingress direction, if $\exists j, (R_{q,j}, iE_{q,j}) \in RQ'_q, e_p^{min} \in iE_{q,j}$, then there may be reachability violation. For $\forall i, iC_{p,i} \in iCs_p$, if $iC_{p,i}.R = R_{q,j}$ and $e_q^{e,min} \notin iC_{p,i}.eE$, then $(e_q^{e,min}, R_{q,j}, iE_{q,j})$ is the part where the reachability is violated and $e_q^{e,min}$ is added to iCs_p for fix. In addition, if $dd_p = true$, and for $\forall i, iC_{p,i} \in iCs_p, iC_{p,i}.R \neq R_{q,j}$, then $(e_q^{e,min}, R_{q,j}, iE_{q,j})$ is the part where the reachability is violated and $(e_q^{e,min}, R_{q,j})$ is added to iCs_p for fix.

We process all the $C' \in \Phi'_C$ according to *Definition 4* to get the processed Φ'_C which has no reachability violation.

Verify if Φ'_I Satisfies Φ'_R and Fix it: For the processed Φ'_R and Φ'_I that have no redundancy, we can verify if Φ'_I satisfies the required reachability of Φ'_R and automatically fix it.

Definition 5: For $I'_p = (e_p^{e,min}, \bigcup(R_{p,i}, iE_{p,i}))$, $RQ'_q = (e_q^{e,min}, \bigcup(R_{q,j}, iE_{q,j}))$, if $e_p^{e,min} = e_q^{e,min}$, then there may be reachability violation. For $\forall j, (R_{q,j}, iE_{q,j}) \in RQ'_q$, if: 1) $\forall i, (R_{p,i}, iE_{p,i}) \in I'_p, R_{q,j} \neq R_{p,i}$, then $(e_q^{e,min}, R_{q,j}, iE_{q,j})$ is the part where reachability is violated and is added to I'_p for fix; 2) $\exists i, (R_{p,i}, iE_{p,i}) \in I'_p, R_{q,j} = R_{p,i}$ and $iE_{q,j} \not\subseteq iE_{p,i}$, then compute $iE_{q,j} - iE_{p,i}$ as the part where reachability is

Algorithm 2: Policy-Level Reachability Verification.

Input: Φ'_I and Φ'_C after *compose*; RQ set Φ_R

Output: processed intent set Φ'_I and constraint set Φ'_C

- 1: Process redundancy within Φ_R to get Φ'_R (same as Φ'_I)
 - /* Verify if Φ'_C satisfies Φ'_R and fix it */
 - 2: for $C' \in \Phi'_C, RQ' \in \Phi'_R$ do
 - 3: if $C'.e^{min} = RQ'.e^{e,min}$ then
 - 4: Use *Definition 4* to process C'
 - /* Verify if Φ'_I satisfies Φ'_R and fix it */
 - 5: for $I' \in \Phi'_I, RQ' \in \Phi'_R$ do
 - 6: if $I'.e^{min} = RQ'.e^{e,min}$ then
 - 7: Use *Definition 5* to process I'
 - 8: if $RQ'.e^{e,min}$ not equal any $I'.e^{e,min}$ in Φ'_I then
 - 9: Use *Definition 5* to process Φ'_I
 - 10: return Φ'_I, Φ'_C
-

Algorithm 3: Intent Compilation.

Input: intent set Φ'_I ; EP set Φ_E

Output: flow rule set Φ_F

- 1: for $ep \in \Phi_E$ do
 - 2: Put ep with the same *label* into the same set $sub\Phi_E$ and get $\Phi_E = \bigcup sub\Phi_E$
 - 3: for $sub\Phi_E \in \Phi_E$ do
 - 4: Use $sub\Phi_E$ as ep_{egress} and find $I' \in \Phi'_I$ whose $e^{e,min}$ matches ep_{egress}
 - 5: Find $sub\Phi_E \in \Phi_E$ that matches $I'.iE$ as $ep_{ingress}$
 - 6: Generate flow rules for each pair of EPs in ep_{egress} and $ep_{ingress}$ and put into Φ_F
 - 7: return Φ_F
-

violated and let $iE_{p,i} = iE_{p,i} \cup iE_{q,j}$ for fix. In addition, if $\exists q, RQ'_q \in \Phi'_R$, for $\forall p, I'_p \in \Phi'_I, e_p^{e,min} \neq e_q^{e,min}$, then RQ'_q is the part where reachability is violated and is added to Φ'_I for fix.

We process all the $I' \in \Phi'_I$ according to *Definition 5* to get the processed Φ'_I which has no reachability violation. The overall implementation of policy-level reachability verification is shown in *Algorithm 2*.

D. Intent Compilation and Deployment

IMS compiles the processed intent set into specific policies that can be accepted by the data plane, which in this paper are OpenFlow flow rules [9]. Intent compilation requires EP's IP address as the unique identifier in the VNet, as well as its location information (the switch and port it is connected to). The compiled flow rules are deployed through the SDN controller to the software switch to which each EP is connected, which implements a default deny mode, and enables whitelisting forwarding behavior through deployed flow rules.

We have processed the redundancy in the intent set in *Section V-B* to obtain $\Phi'_I = \bigcup I'$, where each I' has a different $e^{e,min}$, so that for any EP, if there exists an $I'.e^{e,min}$ that matches it, then that I' is unique. This property will contribute to the efficiency of intent compilation. For each $ep_{egress} \in \Phi_E$, we can find the

unique $I' \in \Phi'_I$ according to its *label*, and find all matching $ep_{ingress}$ according to $I'.iE$. Then use $I'.R$ and EP-related information (IP, switch ID and port) stored in IMS to generate flow rules for each $(ep_{egress}, ep_{ingress})$ pair.

To improve efficiency, the above process can be optimized using hash matching and grouping. First, due to the property of $e^{e,min}$, each ep_{egress} can find a unique $I'.e^{e,min}$ with the same label through hash matching, which only requires traversing Φ_E once without additionally traversing Φ'_I . Similarly, for $e^{i,min} \in iE$, all the matched $ep_{ingress}$ can be found in the same way by traversing iE only once without additionally traversing Φ_E . Second, EPs with the same label can be placed together and matched as a group, further reducing the number of traversals and improving efficiency. The implementation of intent compilation is shown in *Algorithm 3*.

E. Dynamicity Processing

This section details how IMS processes dynamicity, including policy dynamicity and EP dynamicity. For the former, when intents and constraints are dynamically modified via DSL, IMS automatically maintains the required reachability. For the latter, IMS achieves prompt processing through algorithm design and optimizations for dynamically changing EPs.

1) *Policy Dynamicity*: This section details the implementation of the three primitives *delete*, *add* and *maintain* involved in the primitive *modify* for policy dynamicity in the DSL. The combination of *delete* and *maintain* allows the deletion of intents and constraints, and the combination of *add* and *maintain* allows the addition of intents and constraints. Note that for constraint deletion, no reachability violation is caused, i.e., no additional processing is required. Similarly, for intent addition, constraint-based intent composition is simply re-performed without causing the reachability violation. Therefore, we describe the processing of intent deletion and constraint addition in detail below.

Intent Deletion: *delete* can specify the intent set Φ_D or the EPG set $\Phi_{D,E}$, and similarly, *maintain* can specify the intent set Φ_M or the EPG set $\Phi_{M,E}$. If the intent set is specified, IMS processes the redundancy using *Definition 1* to get Φ'_D and Φ'_M , respectively. If the EPG set is specified, IMS automatically matches existing intents against these EPGs to get the intent sets Φ'_D and Φ'_M . After that, we implement *maintain* by trying to keep Φ'_M while deleting Φ'_D from the existing intent set Φ'_I by computing the *difference set* between Φ'_D and Φ'_M . After the processed Φ'_D is obtained, we again compute the *difference set* of Φ'_I and Φ'_D in order to implement *delete*. Therefore, below we define the way to compute the *difference set* between two intent sets that have no redundancy. The implementation of intent deletion is shown in *Algorithm 4*.

Definition 6: For $\Phi'_u = \bigcup I'_u$, $\Phi'_v = \bigcup I'_v$, $\Phi'_u - \Phi'_v$ can be computed as follows. For $\forall p, q$, $I'_{u,p} = (e^{e,min}_p, \bigcup (R_p, iE_p))$, $I'_{v,q} = (e^{e,min}_q, \bigcup (R_q, iE_q))$, if $e^{e,min}_p = e^{e,min}_q$, and for $\forall i$, if $\exists j$, $R_{p,i} = R_{q,j}$, then let $iE_{p,i} = iE_{p,i} - iE_{q,j}$. Finally, we get Φ'_u as the result.

Constraint Addition: *add* can specify the constraint set $\Phi_{A,C}$ and IMS processes the conflict using *Definition 2* to get $\Phi'_{A,C}$.

Algorithm 4: Intent Deletion.

Input: intent set Φ'_I ; $\Phi_D, \Phi_{D,E}, \Phi_M, \Phi_{M,E}$

Output: processed intent set Φ'_I

- 1: **if** *delete*, *maintain* specifies intent sets Φ_D, Φ_M **then**
 - 2: Use **Definition 1** to process redundancy within Φ_D , Φ_M and get Φ'_D, Φ'_M
 - 3: **else if** *delete*, *maintain* specifies EPG sets $\Phi_{D,E}, \Phi_{M,E}$ **then**
 - 4: Use $\Phi_{D,E}, \Phi_{M,E}$ to match intents in Φ'_I and get Φ'_D, Φ'_M
 - 5: Use **Definition 6** to get $\Phi'_D = \Phi'_D - \Phi'_M$
 - 6: Use **Definition 6** to get $\Phi'_I = \Phi'_I - \Phi'_D$ and delete corresponding flow rules
 - 7: **return** Φ'_I
-

Algorithm 5: Constraint Addition.

Input: intent set Φ'_I ; constraint set $\Phi'_{A,C}, \Phi_{A,C}; \Phi_M, \Phi_{M,E}$

Output: processed constraint set Φ'_C

- 1: Use **Definition 2** to process conflict within $\Phi_{A,C}$ and get $\Phi'_{A,C}$
 - 2: **if** *maintain* specifies intent set Φ_M **then**
 - 3: Use **Definition 1** to process redundancy within Φ_M and get Φ'_M
 - 4: **else if** *maintain* specifies EPG set $\Phi_{M,E}$ **then**
 - 5: Use $\Phi_{M,E}$ to match intents in Φ'_I and get Φ'_M
 - 6: **else if** *maintain* specifies “original” **then**
 - 7: Use Φ'_I as Φ'_M
 - 8: Use **Definition 4** to verify whether $\Phi'_{A,C}$ satisfy the reachability requirement of Φ'_M and fix
 - 9: **if** *maintain* does not specify “original” **then**
 - 10: Use **Definition 3** to process conflict between $\Phi'_{A,C}$ and Φ'_I and delete corresponding flow rules
 - 11: Merge $\Phi'_{A,C}$ into Φ'_C and use **Definition 2** to process conflict
 - 12: **return** Φ'_C
-

maintain can specify the intent set Φ_M or the EPG set $\Phi_{M,E}$, or it can specify “original” to represent the original reachability. If the intent set is specified, IMS processes the redundancy using *Definition 1* to get Φ'_M . If the EPG set is specified, IMS automatically matches existing intents against these EPGs to get the intent set Φ'_M . If “original” is specified, IMS treats the existing intent set Φ'_I as Φ'_M directly. After that, we perform the same steps as the reachability verification, i.e., treat Φ'_M as the RQ set and verify that $\Phi'_{A,C}$ satisfies the reachability requirements of Φ'_M and automatically fix it to get the processed $\Phi'_{A,C}$. If *maintain* does not specify “original”, we use *Definition 3* to compose $\Phi'_{A,C}$ and Φ'_I . Finally, we merge $\Phi'_{A,C}$ into the existing constraint set Φ'_C , and use *Definition 2* to process the conflicts in it. The implementation of constraint addition is shown in *Algorithm 5*.

2) *EP Dynamicity*: There are three types of EP dynamicity. **A**: Creating the EP and labeling it; **B**: Destroying the EP; and **C**: Modifying EP’s labels. Type **A** involves the recompilation

of intent, type **B** involves the deletion of flow rules, and type **C** can be considered as a combination of **A** and **B**. Therefore, the processing of **C** is described in detail below.

The procedure of processing EP dynamicity is not complicated. First, we delete the flow rules generated for the EP whose labels have been modified. Then we need to re-perform intent compilation to generate new flow rules for that EP to update its whitelisting access behavior. However, due to the security and business continuity issues that can result from EP status changes, IMS needs to implement prompt processing.

Suppose the EP whose labels have been modified is ep_a . The intuitive way to implement flow rule deletion is to traverse all existing flow rules, find the ones with matching *Src/Dst* IP according to the IP of ep_a and delete them. We can use hash to speed up the above process. Specifically, in intent compilation, we can create a hash-based dictionary D_F whose *Key* is the header field of the flow rule (*Src* IP + *Dst* IP) and *Value* is all the flow rules with the same header field. Another dictionary D_E is also created, whose *Key* is the IP of each EP, while the *Value* stores the header fields of all the flow rules related to that EP. When deleting flow rules, we only need to look up D_E by ep_a , get the header fields of all flow rules related to ep_a and delete them from D_E , and use these header fields as *Key* to get all flow rules that need to be deleted from D_F and delete them from D_F . Note that since ep_a may be used as *Src/Dst* EP in the flow rules, the corresponding header field of the corresponding *Dst/Src* EP in D_E should also be deleted.

We then need to generate new flow rules for ep_a . Since ep_a can have the access allowed by intents to any other EP in VNet, the intuitive way to is to re-perform the intent compilation, but this approach is inefficient. We can optimize the above process based on the idea of *separation*. Suppose the current EP set is Φ_E , then the remaining EP set other than ep_a is $\Phi_E - ep_a$. The flow rules associated with ep_a are two parts, with ep_a as the *Src* EP and ep_a as the *Dst* EP respectively. So we can perform intent compilation in two rounds. In the first round, ep_a is the *Src* EP and $\Phi_E - ep_a$ is the *Dst* EP set. In the second round, $\Phi_E - ep_a$ is the *Src* EP set and ep_a is the *Dst* EP. With the separation approach, the number of EPs involved in intent compilation can be effectively reduced, and intent compilation (Algorithm 3) itself is already optimized so that the efficiency requirement can be met. The implementation of EP dynamicity processing is shown in Algorithm 6.

F. Optimizations

In Sections V-D and V-E2, we have performed optimizations for intent compilation and EP dynamicity. In addition, given the current large-scale nature of VNet, we will describe the optimizations employed by IMS for constraint-based intent composition, policy-level reachability verification, and policy dynamicity to ensure scalable and efficient processing.

Removing Invalid e^{min} : The preprocessing and transformation for EPG in Section IV-C provides the basis for the computability of IMS. However, during preprocessing, IMS adds the full label values to the EPG under unspecified label types, which can result in a large number of invalid e^{min} . In practice,

Algorithm 6: Processing of EP Dynamicity.

Input: intent set Φ'_I ; EP set Φ_E ; modified EP ep_a ; D_F , D_E
Output: set of new flow rules Φ'_F
 /* Delete existing flow rules for ep_a */
 1: Use IP of ep_a to get the value in D_E as *Key* and delete
 2: Use *Key* to get the flow rules corresponding to ep_a in D_F and delete
 /* Generate new flow rules for ep_a */
 3: Use ep_a as the *Src* EP and $\Phi_E - ep_a$ as the *Dst* EP set to perform intent compilation and get $\Phi'_{e,F}$
 4: Use $\Phi_E - ep_a$ as the *Src* EP set and ep_a as the *Dst* EP to perform intent compilation and get $\Phi'_{i,F}$
 5: **return** $\Phi'_F = \Phi'_{e,F} \cup \Phi'_{i,F}$

there is a finite number of labels that EP can be tagged with, i.e., the full set of possible values for the actual EPG is a subset of the EPG space generated by the full set of label types and values. Thus, admin can specify valid or invalid label combinations via IMS. For example, for an EP with label **Label**₀: *Value*₁, admins can specify that for label type **Label**₁, it cannot have label **Label**₁: *Value*₁, or specify that it can only have label **Label**₁: *Value*₀. After that, IMS can remove the invalid e^{min} generated during EPG transformation according to these label combinations, which can effectively reduce the number of e^{min} obtained from EPG transformation. As the number of e^{min} contained in EPG directly affects the computational efficiency of EPG space algebra, therefore, we can effectively optimize the processing efficiency of functionalities in IMS.

Hash-Based Matching and Set Operations. The main computations in the functionality layer include the following parts. *Matching*: find identical subelements from different sets by traversal. *Set operations*: compute the intersection, union, and difference and judge the relationships between EPGs. Hash would be an efficient way to optimize. The hash-based optimizations come from two directions.

1) *Hash-based matching*: For sets Φ_A and Φ_B with n_A and n_B elements respectively, traversal will result in $O(n_A * n_B)$ time complexity when searching for the same subelements in them. To improve the efficiency, the hash value of each subelement can be used as the *Key* and the element as the *Value* to obtain hash dictionaries Φ'_A and Φ'_B . We only need to traverse the *Key* of the hash dictionary with the smaller length and determine if the *Key* is in another hash dictionary ($O(1)$), thus reducing the time complexity to $O(\min(n_A, n_B))$. Therefore, using hash-based matching to find the same R and the same e^{min} in intents, constraints and RQs can effectively improve the processing efficiency.

2) *Hash-based set operations*: For sets Φ_A and Φ_B with n_A and n_B elements e respectively, traversal will result in $O(n_A * n_B)$ time complexity when computing intersection, union, difference or judging relations between them. To improve efficiency, the hash value of each element can be used as its subscript and stored in a new array to obtain hash sets Φ'_A and Φ'_B . The time complexity of hash set operations is shown in Table II. Therefore, using hash-based set operations when

TABLE II
TIME COMPLEXITY OF HASH-BASED SET OPERATIONS

$e \in \Phi_A$	$\Phi'_A \subseteq \Phi'_B$	$\Phi'_A \cap \Phi'_B$	$\Phi'_A \cup \Phi'_B$	$\Phi'_A - \Phi'_B$
$O(1)$	$O(n_A)$	$O(\min(n_A, n_B))$	$O(n_A + n_B)$	$O(n_A)$

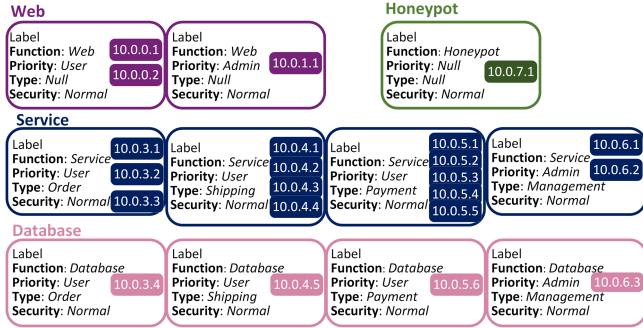


Fig. 6. EPs in the use case and their labels in IMS.

processing EPG space algebra (set operations) can effectively improve the processing efficiency.

VI. USING IMS

This section presents a use case to demonstrate how IMS accomplishes its functionalities to help admins overcome the difficulties and challenges in formulating and maintaining MSG policies. The implementation of IMS for this use case and the experimental environment are described in *Section VII*.

By observing the current cloud application architecture and internal access logic, including VM-based and container-based cloud applications (including microservices), we created and deployed a virtual but rational use case where cloud applications are deployed in different EPs in a distributed manner (Fig. 6). The admin's goal is to provide the cloud application with internal fine-grained and dynamic access control, i.e., only traffic for normal business should be allowed, EPs with security threats or high priority levels should be isolated separately, policies should be verified to prevent impact on normal business, and dynamicity should be processed automatically. Specifically, the security expectations are as follows. *Basic business requirements*: 1) *Web* can access *Service* via TCP 9080. 2) *Service* can access *Database* via TCP 3306. 3) *Compromised* EP can access *Honeypot*. *Security isolation requirements*: 1) *Management Service* can only be accessed by *Admin Web* via TCP 9080. 2) *Database* can only be accessed by *Service* of the same type with it via TCP 3306. 3) *Normal* EPs can only be accessed by *Normal* EPs. After formulating policies based on the above requirements, they should be verified to ensure that the required reachability is not violated. Meanwhile, any possible dynamicity should be processed automatically, e.g., EP status changes.

Current MSG methods (e.g., K8S Network Policy [12]) can meet basic business requirements, but are unable to meet other requirements because they lack critical computability and processing for dynamicity. Therefore, they cannot prevent security isolation requirements from being violated due to policy overlap,

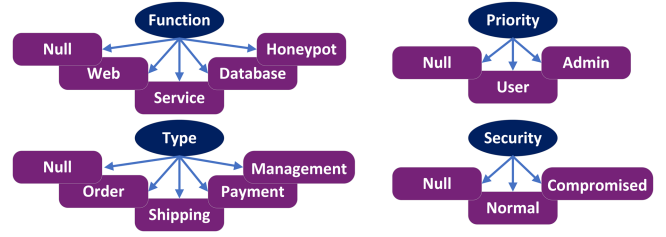


Fig. 7. The label forest for the use case.

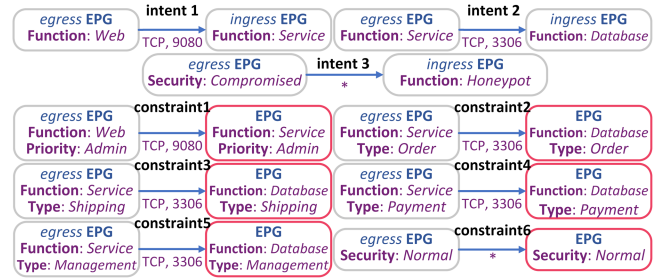


Fig. 8. Intents and constraints for the use case.

cannot verify that policies meet the required reachability, and cannot automatically process dynamicity.

IMS makes it easy to meet security expectations. Whether MSG policies are formulated by a single admin or by multiple admins from different domains, these policies can be composed and verified. First, admins formulate an LF according to business and security requirements (Fig. 7). EPs are then tagged with labels representing their high-level attributes (Fig. 6). After that, admins can easily formulate MSG intents to achieve basic business access, formulate constraints to achieve security isolation requirements (Fig. 8). IMS then performs the constraint-based intent composition, automatically processing any conflicts to prevent the security isolation requirements from being violated due to overlap.

Before policies take effect, admins can verify them to ensure that the required reachability is not violated. Although there are few policies for this use case, there may still be imperceptible reachability violations: the admin requires that any *compromised* EP should be able to access *Honeypot* to collect traffic, but *constraint 6* will only allow any EP with label *Security: Normal* to be accessed only by EPs with label *Security: Normal*, erroneously preventing EPs with label *Security: Compromised* from accessing the honeypot. Note that such reachability violation cannot be found at EP-level due to the semantic gap: Current EPs are all tagged with *Normal*, so reachability violations cannot be observed at EP-level (Fig. 9(a)). This demonstrates the importance of policy-level reachability verification in IMS, which allows admins to formulate RQ to verify if the reachability from compromised EPs to *Honeypot* is satisfied and automatically fix the violation. For this use case, IMS takes 6ms, 2ms and less than 0.1 ms to complete intent composition, verification and translation respectively. The access relationships between EPs allowed by intents are shown in Neo4J (Fig. 9(a)). It can be seen that IMS helps the admins to meet all security expectations.

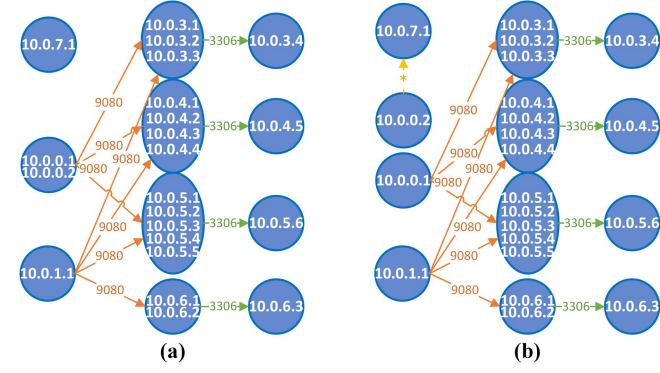


Fig. 9. The access relationships between EPs allowed by intents in Neo4J (the graphs in Neo4J are redrawn here for aesthetics and simplicity).

To demonstrate dynamicity processing in IMS, we simulated the modification of EP labels. Specifically, we modify the label of EP 10.0.0.2 to *Security: Compromised*, indicating that the EP has been compromised (in practice, the label can be automatically modified by IDS using the interface provided by IMS). IMS automatically processes the above dynamicity within 0.1 ms, finds and deletes the flow rules corresponding to this EP in the existing flow rule set and generates the corresponding new flow rules using the optimized intent compilation algorithm. As shown in Fig. 9(b), since the label has changed from *Normal* to *Compromised*, this EP is only acted upon by *intent 3*, so all its existing connections are removed and its traffic can only access the *Honeypot* (10.0.0.7).

Summary: We use the above use case to demonstrate the functionalities of IMS. Admins' security requirements are met, with any conflicts and reachability violations automatically detected and processed. Meanwhile, IMS processes dynamicity automatically and promptly, ensuring that admins' security expectations always remain in effect.

VII. EVALUATION

We evaluate IMS from the following perspectives. *Q1*: Is the performance of the functionality layer and intent compilation efficient and scalable? *Q2*: Do the optimizations for the functionality layer meet expectations? *Q3*: Is the processing of EP dynamicity sufficiently prompt?

A. System Implementation and Experimental Setup

Implementation: We implement a prototype of IMS using Python. IMS is adapted to the famous and widely used open source SDN controller ONOS [26], and takes OpenFlow [9] flow rules as the carrier to realize MSG through ONOS northbound and southbound interfaces. Specifically, IMS retrieves and stores the information such as the IP addresses of EPs and switch IDs and port that EPs are connected to from the interface *onos/v1/hosts*, and deploys flow rules from the interface *onos/v1/flows*. A key advantage of choosing SDN architecture is that it can be used as a unified way to build and manage virtual networks (data planes) to accommodate both container clouds and VM clouds. In addition, one of the reasons for choosing

ONOS as the SDN controller is its richness of interfaces and adaptability: we can realize the connection between ONOS and cloud managers OpenStack [23] and Kubernetes [24] through SONA project [27]. The creation and modification of VNets and the creation, operation, and destruction of EPs are all completed by the cloud manager, and the corresponding information are captured by ONOS and further obtained by IMS. Note that IMS can also be connected directly to the cloud manager through the corresponding interfaces. Neo4J [28] is chosen as the graph database to store and display the access relationships between EPs.

Experimental Setup: In this experiment, IMS runs as a single-threaded Python program, deployed alongside Neo4J on a PC with a Core i7 2.6 GHz CPU and 16 GB memory. ONOS is deployed on a server with 2 * Xeon 2.2 GHz CPU and 128 GB memory. For the sake of simplicity, this experiment simulates the creation of a virtual network based on the software switch Open vSwitch (OVS) [29] via Mininet [30], and simulates the creation of EPs and their connection to OVS. Mininet runs on the same server as ONOS.

Dataset: We created datasets for performance evaluation. First, a label forest (LF) containing 4 label trees (LTs) is randomly generated, and each LT has 5 leaf nodes. This LF can express about $2^{20} = 1048576$ kinds of EPGs, and more MSG intents and constraints can be expressed by the combination of EPGs. We believe that this number is sufficient to satisfy the vast majority of MSG needs. Based on this LF, we randomly generate a DSL-form dataset *DS1* containing 2000 intents, 2000 constraints, and 2000 reachability queries (RQs). In addition, to test the processing of policy dynamicity, we generated an additional random dataset *DS2* containing 1000 constraints, 1000 intents and 1000 EPGs in the same way as above. Finally, to test the intent compilation, we used Mininet to generate 1000 EPs (this number is sufficient to satisfy the vast majority of MSG needs) with different IPs, and randomly tagged each EP with labels using the above LF.

B. Results and Discussion

We optimize the functionality layer of IMS using the optimizations in Section V-F. There are two types of optimizations: **A**: Removing invalid e^{min} , and **B**: Hash-based optimization. We use **A** and **!A**, **B** and **!B** to represent the adoption/non-adoption of optimizations **A** and **B** respectively. For optimization **A**, we randomly specified invalid combinations of labels, which in the best case would reduce the number of e^{min} obtained from the EPG conversion to 20% of the original. Note that even when optimization **B** is not used (i.e., **!B**), we still use hash-based set operations that would otherwise fail to complete the computation in an acceptable time. Nevertheless, optimization **B** is still very efficient, which will be demonstrated below.

Constraint-Based Intent Composition: We compared different ways of optimization combinations of (**!A**, **!B**), (**!A**, **B**), (**A**, **!B**) and (**A**, **B**). For each combination, we set the number of intents and constraints from *DS1* starting at 400 and increasing by 400 each round up to 2000. Each round was run 10 times and the mean and standard deviation were calculated. The results are

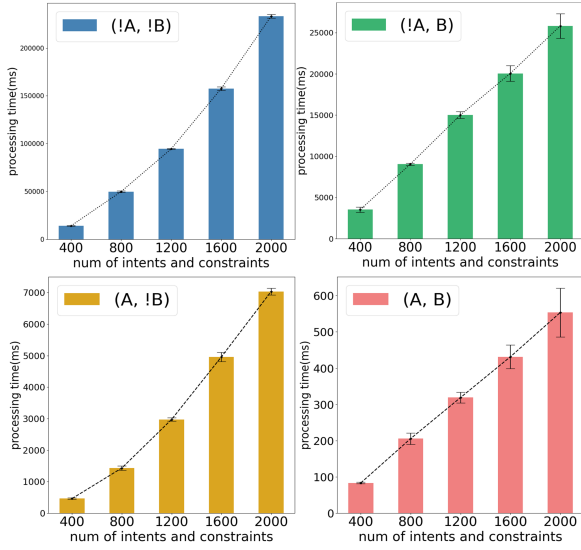


Fig. 10. Experimental results for constraint-based intent composition.

TABLE III
COMPARISON OF OPTIMIZATIONS FOR CONSTRAINT-BASED INTENT COMPOSITION WITH NUMBER OF 2000 (WITH (A, B) AS THE BASE)

(!A, !B)	(!A, B)	(A, !B)	(A, B)
422x	47x	13x	553.4ms

shown in Fig. 10. First, the time consumption for performing intent composition all decreases significantly as the optimization combination moves from (!A, !B) to (A, B). Furthermore, it can be seen that Optimization B is able to change the time consumption from non-linear to linear as the number of intents and constraints increases, which adequately achieves the objective of hash-based optimization. Optimization A, on the other hand, effectively reduces the time consumption by reducing the number of invalid e^{min} . Table III also compares the multipliers of the time consumption for the other combinations when the intent and constraint are 2000, with the combination (A, B) as the base. Therefore, our optimizations are effective. Second, for the optimization combination (A, B), the time consumption grows linearly with increasing number of intents and constraints, which illustrates the scalability of this functionality in IMS.

Policy-Level Reachability Verification: Similarly, we compared different ways of optimization combinations of (!A, !B), (!A, B), (A, !B), and (A, B). For each combination, we set the number of intents, constraints and reachability queries (RQs) from *DS1* starting at 400 and increasing by 400 each round up to 2000. In each round, we perform intent composition first and use the processed intent and constraint sets for verification (we only record the time of verification). Each round was run 10 times and the mean and standard deviation were calculated. The results are shown in Fig. 11 and Table IV. Similar to the conclusions obtained from the experimental result of intent composition, our optimizations are effective and this functionality is scalable.

Policy Dynamicity: Two tests are included: intent deletion and constraint addition. For intent deletion, we use the intents in *DS2* as the intent set to be deleted specified by primitive *delete*

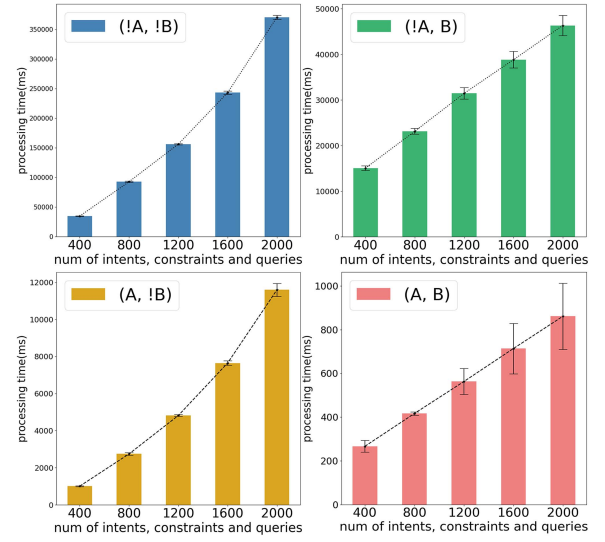


Fig. 11. Experimental results for policy-level reachability verification.

TABLE IV
COMPARISON OF OPTIMIZATIONS FOR POLICY-LEVEL REACHABILITY VERIFICATION WITH NUMBER OF 2000 (WITH (A, B) AS THE BASE)

(!A, !B)	(!A, B)	(A, !B)	(A, B)
430x	54x	13x	861.0ms

TABLE V
COMPARISON OF OPTIMIZATIONS FOR INTENT DELETION WITH NUMBER OF 1000 (WITH (A, B) AS THE BASE)

(!A, !B)	(!A, B)	(A, !B)	(A, B)
4.4x	1.9x	1.3x	3111.2ms

and the EPGs in *DS2* as the EPG set specified by primitive *maintain*. For constraint addition, we use the constraints in *DS2* as the constraint set to be added as specified by primitive *add*, and primitive *maintain* specifies “original”, indicating that the original reachability is maintained, which will result in the greatest amount of computation. For both tests above, we compared different ways of optimization combinations of (!A, !B), (!A, B), (A, !B), and (A, B). Before performing the above two tests, we first perform intent composition on 1000 intents and 1000 constraints from *DS1* and used 1000 RQs for reachability verification to obtain the original intent and constraint sets on which both of the above tests were performed. For intent deletion, we set the number of intents specified by primitive *delete* to start at 200, and the number of EPGs specified by primitive *maintain* to be one tenth of that (i.e., starting at 20), increasing by 200 (20) each round up to 1000 (100). For constraint addition, we set the number of constraints specified by primitive *add* to start at 200 and increase by 200 each round up to 1000. Since primitive *maintain* is specified as “original”, it actually specifies the original intent set whose number is fixed at 1000. Each round was run 10 times and the mean and standard deviation were calculated. The results are shown in Figs. 12, 13 and Tables V, VI. It can be seen that

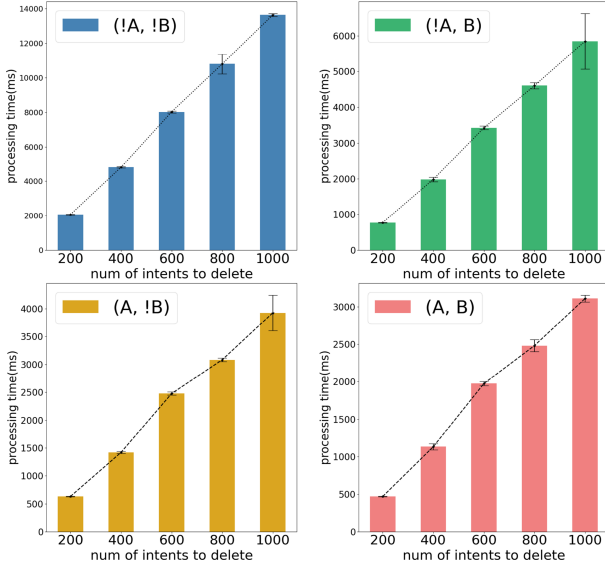


Fig. 12. Experimental results for intent deletion.

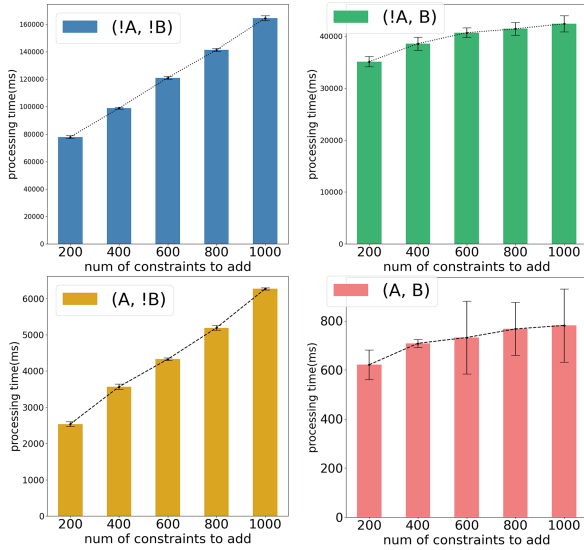


Fig. 13. Experimental results for constraint addition.

TABLE VI
COMPARISON OF OPTIMIZATIONS FOR CONSTRAINT ADDITION WITH NUMBER OF 1000 (WITH (A, B) AS THE BASE)

(!A, !B)	(!A, B)	(A, !B)	(A, B)
211x	54x	8x	781.4ms

our optimizations are effective and that the processing of policy dynamicity is scalable. In addition, it is worth noting that for constraint addition, since *maintain* specifies “original”, and the number of the original intent set is fixed, the growth curve of time consumption becomes progressively flatter as the number of constraints increases with the use of Optimization B.

Intent Compilation: We set the number of intents and constraints from *DSI* to start at 200 and increase by 200 each round up to 1000. In each round, we first perform intent composition

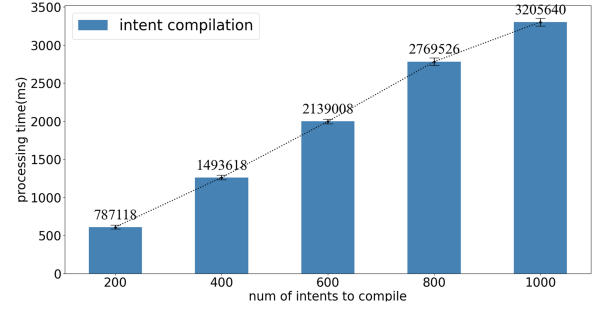


Fig. 14. Experimental results for intent compilation (the number on the bar chart is the number of flow rules generated).

TABLE VII
EXPERIMENTAL RESULTS FOR EP DYNAMICITY

Processing time (mean (std)) (ms)	Generated flow rules (mean (std))
5.05 (0.49) ms	6378.0 (583.3)

to obtain the processed intent set, and then perform intent compilation using this intent set and the EP set from *DSI* (number 1000), running each round 10 times and calculating the mean and standard deviation. The results are shown in Fig. 14. In a VNet with a large number of EPs, IMS is still able to perform intent compilation to generate a large number of flow rules with high efficiency (compile and generate 10^6 of flow rules in 1000 ms), which benefits from the redundancy processing in intent composition and the optimizations for intent compilation. It can also be seen that the time consumption of intent compilation increases linearly as the number of intents increases, which reflects its scalability.

EP Dynamicity: We first perform intent composition on 1000 intents and 1000 constraints from *DSI* to obtain the processed intent set, and use this intent set and the EP set from *DSI* (number 1000) to perform intent compilation to obtain the original flow rule set. We then randomly modify the label of each EP in the EP set, for which IMS will perform dynamicity processing. The above operation is performed 10 times for each EP and the average value is used as the time consumption for that EP. We performed the above operation for all 1000 EPs in the EP set and calculated the mean and standard deviation. Note that we only record the processing time of IMS, excluding the processing latency of the SDN controller and switches. The results are shown in Table VII. Even with a large number of intents and EPs, IMS is still able to complete dynamicity processing in about 5.05 ms, removing about 6000 original flow rules and generating about 6000 new flow rules promptly. It is worth noting that the number of flow rules of 6000 is huge for a single EP and will be much smaller in a real scenario [31], because in practice the number of other EPs that an EP can access is limited, unlike in this experiment where an EP can access almost all other EPs due to the random generation of the dataset. And the processing time for EP dynamicity of IMS is positively related to the number of flow rules. Therefore, with a huge number of flow rules, the processing time of IMS for EP dynamicity is small enough to meet the requirement for prompt and efficient processing.

We also compare IMS with the label-based MSG method eZTrust [16], which, to the best of our knowledge, is the only work that considers improving the efficiency of MSG policy enforcement (which is related to EP dynamicity). eZTrust is for container clouds and uses eBPF to tag and authenticate packets and uses a fast and slow path approach to optimize local authentication efficiency. However, when EP dynamicity occurs (i.e., its label is modified), eZTrust lacks corresponding optimizations and must perform slow path processing, i.e., request the label mapping for the packet and perform a slow policy matching process, which is similar to other eBPF-based MSG, e.g., Cilium [14]. In contrast, IMS is significantly more efficient than eZTrust benefiting from redundancy processing based on the computability and several optimizations. Under similar conditions and hardware performance to the eZTrust experiment (4 EPs and 60 policies in total, modifying the labels of 3 EPs), the processing time of IMS is <0.1 ms, while the processing time of eZTrust is around 1 ms. In addition, the compilation approach used by IMS is more pervasive and can be used in both VM and container clouds with no kernel requirements, while eZTrust is only available for container clouds and has a high kernel version requirement. Finally, IMS processes EP dynamicity by actively generating all the flow rules associated with that EP at once. In contrast, eZTrust must perform the above slow path processing passively once for each packet miss (corresponding to a flow rule).

Summary: From the above experimental results for the functionality layer and intent compilation of IMS, we can answer *Q1* and *Q2*. First, the time consumption of the functionality layer and intent compilation grows linearly, which effectively illustrates the scalability of IMS. Second, the effectiveness of our proposed optimizations is effectively demonstrated by comparing different combinations of optimizations for the functionality layer. And from the above experimental results for EP dynamicity, we can answer *Q3*. With a large number of intents and EPs, IMS has a small enough processing time to meet the requirements for prompt processing.

VIII. DISCUSSION

The Effectiveness of MSG: IMS implements MSG by compiling the intents into flow rules for the data plane. If problems of flow rule conflict or the inconsistency between the control plane and the data plane occur, the flow rules used for MSG may become invalid. These problems have been studied extensively recently, which is beyond the scope of IMS. There are now many solutions to the above problems, such as VeriFlow [32]. Another common problem with SDN is the explosion of flow rules. As the number of EPs in the VNet increases, the number of flow rules increases dramatically. Hardware SDN switches have a small capacity and therefore struggle to support large-scale flow rules. However, in the design and implementation of IMS, we use the software switch (e.g., OVS [29]) as the policy enforcement point. Unlike hardware switches, software switches are not affected by the explosion problem due to the larger memory of the host.

Layer 7 (L7) MSG Policy: As OpenFlow flow rules cannot support the semantics associated with L7 traffic (only Layers

2-4 are supported), IMS does not currently support MSG policies for L7. Support for L7 policies requires additional policy enforcement, such as using the agent in the sidecar model [14]. However, for IMS to support L7 policies, it is only necessary to extend the semantics of R (relation) in the DSL, leaving the rest of the semantics and functionality unchanged, and to use an enforcement method that supports L7 policies. We have made the support for L7 policies our future work.

In fact, the formulation of L7 MSG policies is more challenging and better reflects the requirement for the computability in IMS. L7 policy is a refinement of L4 policy, e.g., specifying further access permissions for methods such as *Get* and *Post* over TCP port 80 of the HTTP protocol. This leads to further policy overlap issues, e.g., if the admin wants to formulate a strict L7 policy, but this policy is overlapped by an L4 policy with the overlapping EPG and the same protocol and port, then this L7 policy will fail, leading to the access scope being erroneously extended, which is clearly against the admin's intent. Although some cloud-native solutions (e.g., Cilium [14]) support L7 policies, they are unable to address the above problems due to the lack of computability to compute policy overlap. With the support for L7 policies, IMS can solve these problems through constraint-based intent composition.

Extending the Functionality Layer of IMS: The functionality layer of IMS is based on the computability provided by the EPG space and algebra. The functionality layer is therefore extensible. For example, while whitelisting MSG policies containing only *allow* actions provide better security, in some scenarios a blacklisting approach containing *deny* actions provides better convenience. However, the addition of *deny* creates explicit policy conflicts, i.e., more than two policies with different actions are applied to the same EP. Based on the computability of IMS, new functionality can be easily customized to detect and process the above conflicts at policy-level. In the future, we would like to package the functions related to the computability in IMS. Developers can customize functionalities such as policy conflict processing and verification for specific scenarios, define them using a similar approach as in Section V, and easily develop them using packaged functions. In summary, based on the computability, IMS can be extended in the future to include more functionalities at the functionality layer according to actual requirements.

IX. RELATED WORK

Micro-Segmentation: As one of the primary security methods for the VNet in the cloud, MSG has received extensive research. The related work can be divided into three categories: 1) IP-based MSG [6], [7], [8], 2) Group-based MSG [10], [11], and 3) Label-based MSG [12], [13], [14], [15], [16]. IP-based MSG formulates MSG policies based on IP with SDN flow rules as the carrier. However, VNet is highly dynamic and requires a large number of MSG policies, and IP-based MSG policies are too complex to formulate and maintain. Group-based MSG divides EPs into several groups and formulates inter-group access policies, which reduces the number of MSG policies in a group manner, but still faces the same problems as IP-based

MSG. Label-based MSG tags EPs with the labels that express high-level attributes, and formulates MSG policies through labels, which reduces the difficulty of policy formulation and maintenance.

Currently, label-based MSG [12], [13], [14], [15], [16] is widely used as a paradigm. However, they are still faced with the problems of policy inconsistency and reachability violation, and cannot provide comprehensive dynamicity processing. In contrast, IMS implements functionalities to address the problems of policy inconsistency and reachability violation based on the computability provided by the EPG space and algebra, and implements comprehensive and prompt dynamicity processing.

Policy Generation: Some studies [33], [34], [35] attempt to automatically obtain the calling relationships between services to generate access control policies. The most representative methods are AUTOARMOR [31], which automatically generates MSG policies by extracting the calling relationships in the microservice source code, and NeuVector [36], which automatically generates MSG policies from historical traffic of EPs. However, due to non-negligible shortcomings, these methods failed to become the mainstream MSG methods such as Kubernetes Network Policy [12]. For example, AUTOARMOR will fail if the source code is not available; NeuVector cannot guarantee that the EP is not malicious when collecting traffic; and none of them can guarantee the completeness of the captured access relationships and therefore cannot generate complete policies. In fact, these methods are complementary to IMS. First, the reachability verification capability of IMS can compensate for their lack of policy completeness. Second, there are still overlaps between generated policies, which can be resolved based on the computability of IMS.

Policy Composition: Some studies attempt to compose policies, such as Frenetic [37] and Pyretic [38], which serially and parallelly compose access control and service chain policies, respectively. The representative policy composition method PGA [39] comes closest to IMS. PGA provides constraint primitives for policies and achieves some degree of automatic conflict processing via constraints in composition. IMS draws on the idea that PGA introduces constraints to realize policy composition. However, compared to PGA: 1) IMS focuses on the implementation of MSG in the cloud VNet, while PGA is used for physical network access control and service function chain in enterprises or data centers. 2) While PGA requires pre-specified overlap relationships between policies manually via *label mapping*, IMS, based on EPG space and algebra, can automatically compute all explicit and implicit overlaps between policies and provide complete set operations. 3) While PGA simply specifies the exclusivity of the EPG, IMS provides finer-grained constraint semantics and can automatically detect and process conflicts between constraints.

Policy Verification: Recently, network policy verification has received considerable attention and research. HSA [22] is one of the pioneers of policy verification, enabling static verification of network policies by formally representing the packet header field and defining algebraic operations. VeriFlow [32] goes a step further by verifying in real time whether the policy change

affects the expected invariants. However, the above work and subsequent derivations and extensions are implemented at the device and the EP level, as is the case with IP-based MSG, and these methods are no longer applicable to MSG policy verification of the VNet in the cloud.

PDN [40] is representative of current work in the area of high-level abstraction network policy verification. However, compared to IMS: 1) PDN focuses on physical networks, while IMS focuses on the VNet of the cloud with a larger number of EPs and greater dynamicity, which leads to additional challenges, including higher abstraction for policy formulation and verification, and dynamicity processing. 2) PDN implements policy verification using metagraphs. However, this approach places EPs into groups (similar to group-based MSG) as nodes of the graph, and the set relationships between nodes must be specified and related to the actual underlying network, thus achieving only a partial decoupling of the policy from the underlying network. In contrast, IMS has a higher level of abstraction and automation: policies are formulated based on EPGs that are flexibly specified by labels with high-level abstraction semantics, and policy verification can be automatically implemented through the EPG space and algebra, and is not affected by the underlying network. 3) PDN does not consider dynamicity. In contrast, the VNet of the cloud that IMS focuses on has high dynamicity and IMS achieves comprehensive and prompt dynamicity processing.

Dynamicity Processing: In the MSG scenario, eZTrust [16] is a representative work that considers dynamicity processing. In Section VII, we have compared eZTrust and IMS in terms of adaptability and processing efficiency: IMS is more adaptable and has a higher dynamicity processing efficiency.

X. CONCLUSION

We propose IMS, the first intent-driven MSG system towards computability and dynamicity. IMS innovatively defines the EPG space and algebra to provide computability, thus solving the problems of policy inconsistency and reachability violation. Meanwhile, IMS implements comprehensive and prompt dynamicity processing. Performance evaluations show that IMS can promptly process EP dynamicity, and the functionalities of IMS are scalable in terms of performance. For future work, we are considering further extensions to the DSL of IMS and the use of a new policy enforcement approach to support L7 MSG policies; and the packaging of functions related to computability to enable customization and on-demand extension of the functionality layer of IMS.

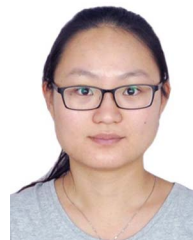
REFERENCES

- [1] J. Kindervag, "Build security into your network's DNA: The zero trust network architecture," 2010. [Online]. Available: <https://media.paloaltonetworks.com/documents/Forrester-Build-Security-Into-Your-Network.pdf>
- [2] J. Turner, "Zero trust explained: The ultimate guide to zero trust security," 2024. [Online]. Available: <https://www.strongdm.com/ebooks/zero-trust-ebook>
- [3] NIST, "Zero trust architecture," 2020. [Online]. Available: <https://www.nist.gov/publications/zero-trust-architecture>
- [4] A. Aldribi, "Documentation for the ISOT cloud intrusion detection dataset," 2020. [Online]. Available: <https://onlineacademiccommunity.uvic.ca/isot/2022/11/25/cloud-security-datasets/>

- [5] B. Arzani et al., "PrivateEye: Scalable and privacy-preserving compromise detection in the cloud," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, San Jose, CA, USA, 2020, pp. 797–815.
- [6] ONF, "ONOS intent framework," 2016. [Online]. Available: <https://wiki.onosproject.org/display/ONOS/Intent+Framework>
- [7] Y. Rodrigues, "NIC architecture," 2018. [Online]. Available: <https://wiki-archive.opendaylight.org/images/8/8b/Nic-arch-conversation.pdf>
- [8] Huawei, "Nemo," 2017. [Online]. Available: <https://wiki-archive.opendaylight.org/view/NEMO:Main>
- [9] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008.
- [10] OpenStack, "Neutron security groups," 2014. [Online]. Available: <https://wiki.openstack.org/wiki/Neutron/SecurityGroups>
- [11] S. Naiksatam, "Group based policy," 2018. [Online]. Available: <https://wiki.openstack.org/wiki/GroupBasedPolicy>
- [12] Kubernetes, "Network policies," 2024. [Online]. Available: <https://kubernetes.io/zh/docs/concepts/services-networking/network-policies/>
- [13] Illumio, "Illumio core design guide," 2023. [Online]. Available: <https://docs.illumio.com/core/21.1/Content/Home.htm>
- [14] Isovalent, "Cilium network policy," 2024. [Online]. Available: <https://docs.cilium.io/en/stable/security/policy/>
- [15] Tigera, "Calico network policy," 2023. [Online]. Available: <https://docs.tigera.io/calico/3.25/reference/resources/networkpolicy>
- [16] Z. Zaher, H. Chang, S. Mukherjee, and J. Van der Merwe, "eZTrust: Network-independent zero-trust perimeterization for microservices," in *Proc. ACM Symp. SDN Res.*, New York, NY, USA, 2019, pp. 49–61.
- [17] M. Bishop, *Computer Security: Art and Science*. Reading, MA, USA: Addison-Wesley Professional, 2002.
- [18] Cloud Security Alliance, "CISO perspectives and progress in deploying zero trust," 2022. [Online]. Available: <https://cloudsecurityalliance.org/artifacts/ciso-perspectives-and-progress-in-deploying-zero-trust/>
- [19] Gartner, "Innovation insight: Intent-based networking systems," 2017. [Online]. Available: <https://www.gartner.com/en/documents/3599617>
- [20] Amazon, "Amazon VPC," 2024. [Online]. Available: <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- [21] CISA, "Indicators associated with WannaCry Ransomware," 2018. [Online]. Available: <https://www.cisa.gov/news-events/alerts/2017/05/12/indicators-associated-wannacry-ransomware>
- [22] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *Proc. 9th USENIX Symp. Netw. Syst. Des. Implementation*, San Jose, CA, USA, 2012, pp. 113–126.
- [23] OpenStack, "OpenStack wiki," 2018. [Online]. Available: https://wiki.openstack.org/wiki/Main_Page
- [24] The Linux Foundation, "Production-grade container orchestration kubernetes," 2024. [Online]. Available: <https://kubernetes.io/>
- [25] ISO/IEC, "Information technology – syntactic metalanguage – extended BNF," 1996. [Online]. Available: <http://standards.iso.org/itf/PubliclyAvailableStandards/>
- [26] ONF, "ONOS wiki," 2020. [Online]. Available: <https://wiki.onosproject.org/>
- [27] ONF, "ONOS SONA," 2018. [Online]. Available: <https://wiki.onosproject.org/display/ONOS/SONA%3A+DC+Network+Virtualization>
- [28] Neo4J, "Neo4j," 2024. [Online]. Available: <https://neo4j.com/>
- [29] The Linux Foundation, "Open vSwitch," 2016. [Online]. Available: <http://www.openvswitch.org/>
- [30] Mininet, "Mininet," 2022. [Online]. Available: <http://mininet.org/>
- [31] X. Li, Y. Chen, Z. Lin, X. Wang, and J. H. Chen, "Automatic policy generation for inter-service access control of microservices," in *Proc. 30th USENIX Secur. Symp.*, Lombard, IL, USA, 2021, pp. 3971–3988.
- [32] A. Khurshid, X. Zou, W. Zhou, M. Caesar, and P. B. Godfrey, "Veriflow: Verifying network-wide invariants in real time," in *Proc. 10th USENIX Conf. Netw. Syst. Des. Implementation*, Lombard, IL, USA, 2013, pp. 15–28.
- [33] A. Saadaoui and L. Scott, "Web services policy generation based on SLA requirements," in *Proc. IEEE 3rd Int. Conf. Collaboration Internet Comput.*, Los Alamitos, CA, USA, 2017, pp. 146–154.
- [34] X. Xiao, A. Paradkar, S. Thummalapenta, and T. Xie, "Automated extraction of security policies from natural-language software documents," in *Proc. ACM SIGSOFT 20th Int. Symp. Found. Softw. Eng.*, New York, NY, USA, 2012, pp. 1–11.
- [35] C. Xiang et al., "Towards continuous access control validation and forensics," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2019, pp. 113–129.
- [36] NeuVector, "Neuvector," 2024. [Online]. Available: <https://neuvector.com/>
- [37] N. Foster et al., "Frenetic: A network programming language," in *Proc. 16th ACM SIGPLAN Int. Conf. Funct. Program.*, New York, NY, USA, 2011, pp. 279–291.
- [38] C. Monsanto, J. Reich, N. Foster, J. Rexford, and D. Walker, "Composing software defined networks," in *Proc. 10th USENIX Symp. Netw. Syst. Des. Implementation*, Lombard, IL, USA, 2013, pp. 1–13.
- [39] C. Prakash et al., "PGA: Using graphs to express and automatically reconcile network policies," in *Proc. ACM Conf. Special Int. Group Data Commun.*, New York, NY, USA, 2015, pp. 29–42.
- [40] D. Ranathunga, M. Roughan, and H. Nguyen, "Verifiable policy-defined networking using metagraphs," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 482–494, Jan./Feb. 2022.



Zixuan Ma received the BS degree in computer science and technology from the Nanjing University of Information Science and Technology, Jiangsu, China, in 2019. He is currently working toward the PhD degree with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS). His research interests include network security, intent-based networks, software-defined networking, and micro-segmentation.



Chen Li received the PhD degree in computer system architecture from the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS) in 2023. She is currently an engineer with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS). Her current research interest include cloud data center security.



Yuqi Zhang received the BS degree in electronic information engineering from Shandong Normal University, Jinan, China, in 2019. She is currently working toward the PhD degree with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS). Her research interests include network security, software-defined networking, and network protocol stack optimization.



Ruibang You received the PhD degree in computer system architecture from the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS) in 2020. He is currently a senior engineer with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS). His current research interests include system security and cloud data centre network security.



Bibo Tu received the PhD degree in computer system architecture from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS) in 2009. He is currently a researcher with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), and a professor with the School of Cyber Security, University of Chinese Academy of Sciences (UCAS). His current research interests include cloud data center security, trusted computing, and software defined security.